CODE4LIFE TECHNOLOGY

LƯU LẠI ĐỂ ÔN THI

CẨM NANG KTMT

KIẾN TRÚC MÁY TÍNH VÀ HỢP NGỮ • 128 TRANG

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        s += a[i][j];       // 0.010 giây

for (j = 0; j < N; j++)
    for (i = 0; i < N; i++)
        s += a[i][j];       // 0.247 giây
```

Đổi chiều duyệt, chậm gấp 24 lần. Vì sao?

Dữ liệu | MIPS | x86-64 | Pipeline | Cache

Code4Life Technology - Tào Việt Đức

Bản đồ cẩm nang

Đi từ bit và cổng logic, qua hợp ngữ, tới bộ xử lý, cache và ôn thi.

1 TỔNG QUAN Trang 3

2 BIỂU DIỄN DỮ LIỆU Trang 14

3 MẠCH SỐ Trang 26

4 HỢP NGỮ MIPS Trang 35

5 HỢP NGỮ X86-64 Trang 55

6 BỘ XỬ LÝ Trang 71

7 BỘ NHỚ VÀ CACHE Trang 84

8 VÀO RA VÀ SONG SONG Trang 95

9 THỰC HÀNH Trang 103

10 ÔN THI Trang 111

Dùng cẩm nang này thế nào?

Môn này khó vì trừu tượng. Cẩm nang cho bạn chạy thử mọi thứ: từ cổng logic, hợp ngữ đến cache.

Mục tiêu của bạn	Nên đọc
Qua môn Kiến trúc máy tính	**Phần 1 đến 4, 6, 7, rồi 10**
Học môn Hợp ngữ	**Phần 2, 4, 5**
Sắp thi cuối kỳ	**Phần 10, rồi phần 6, 7**
Viết code chạy nhanh hơn	**Phần 6, 7, 8**
Đọc được mã máy, debug sâu	**Phần 5, 9**

Dòng ghi trên kết quả	Ý nghĩa
KẾT QUẢ KHI CHẠY	Python, chạy được trên mọi máy
BỘ GIẢ LẬP MIPS	Chạy bằng mips.py kèm theo, nhiều trường dùng MARS
CHẠY TRÊN LINUX	Hợp ngữ NASM chạy thật trên Linux x86-64
HỢP NGỮ DO GCC SINH RA	Lệnh gcc -S thật, theo quy ước Linux

Nội dung bám theo Patterson và Hennessy, tài liệu MARS, NASM và gcc. Mỗi trường có đề cương riêng, hãy so lại với chương trình của bạn.

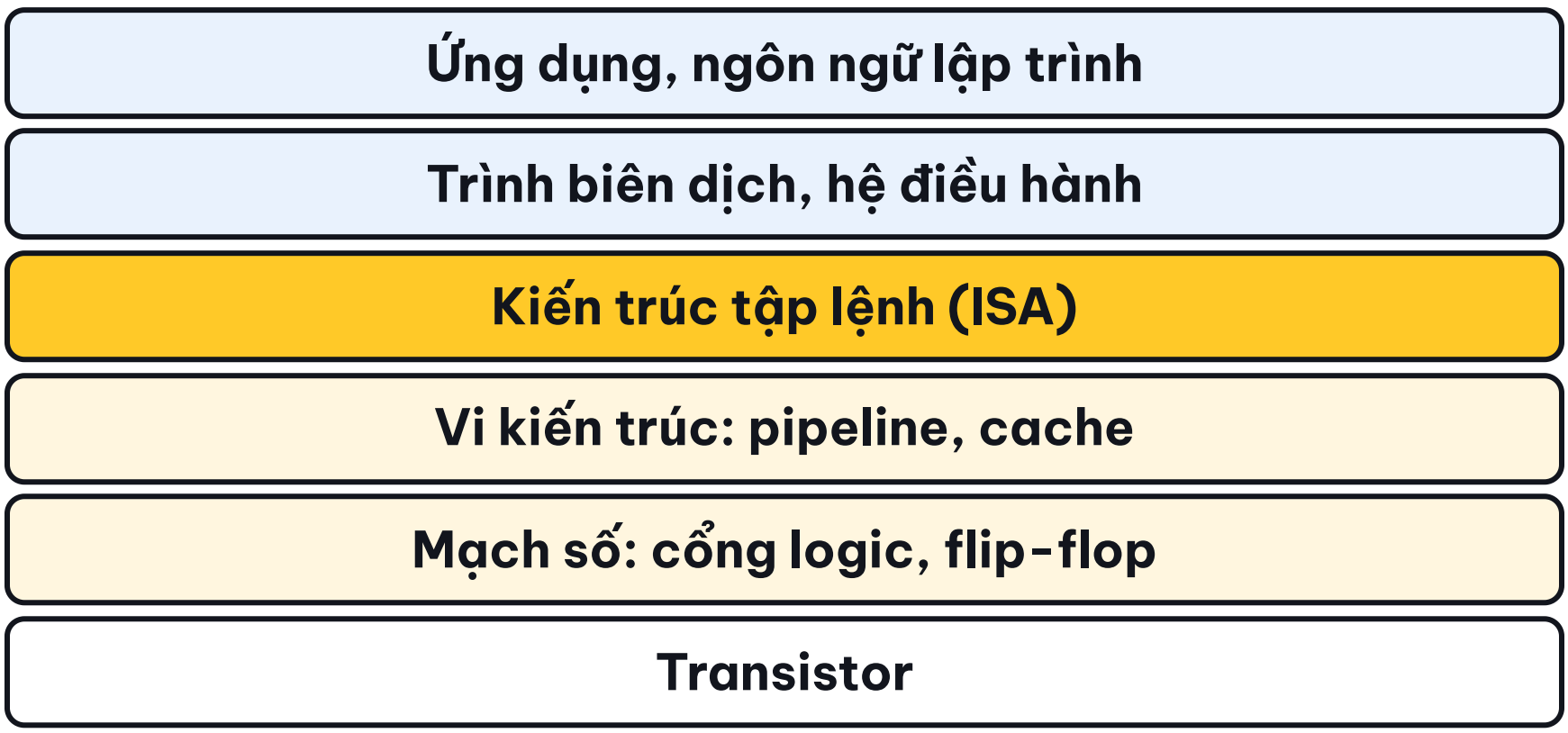
Một khái niệm, nhiều tên gọi

Tiếng Anh	Các tên hay gặp trong giáo trình Việt Nam
Instruction set	Tập lệnh
Register	Thanh ghi
Assembly, assembler	Hợp ngữ, trình hợp dịch
Two's complement	Số bù 2
Overflow	Tràn số
Datapath	Đường dữ liệu
Control unit	Khối điều khiển
Pipeline	Đường ống
Hazard	Xung đột, rủi ro
Forwarding	Chuyển tiếp dữ liệu
Stall	Dừng, chèn bong bóng
Branch	Rẽ nhánh
Cache hit, miss	Trúng, trượt
Throughput, latency	Thông lượng, độ trễ

Đề thi có thể dùng tên khác với giáo trình bạn học. Hiểu bản chất thì đổi tên vẫn làm được.

1 Kiến trúc máy tính học gì?

Máy tính là nhiều tầng xếp chồng. Môn này nối từ dòng code bạn viết xuống tới dòng điện trong chip.

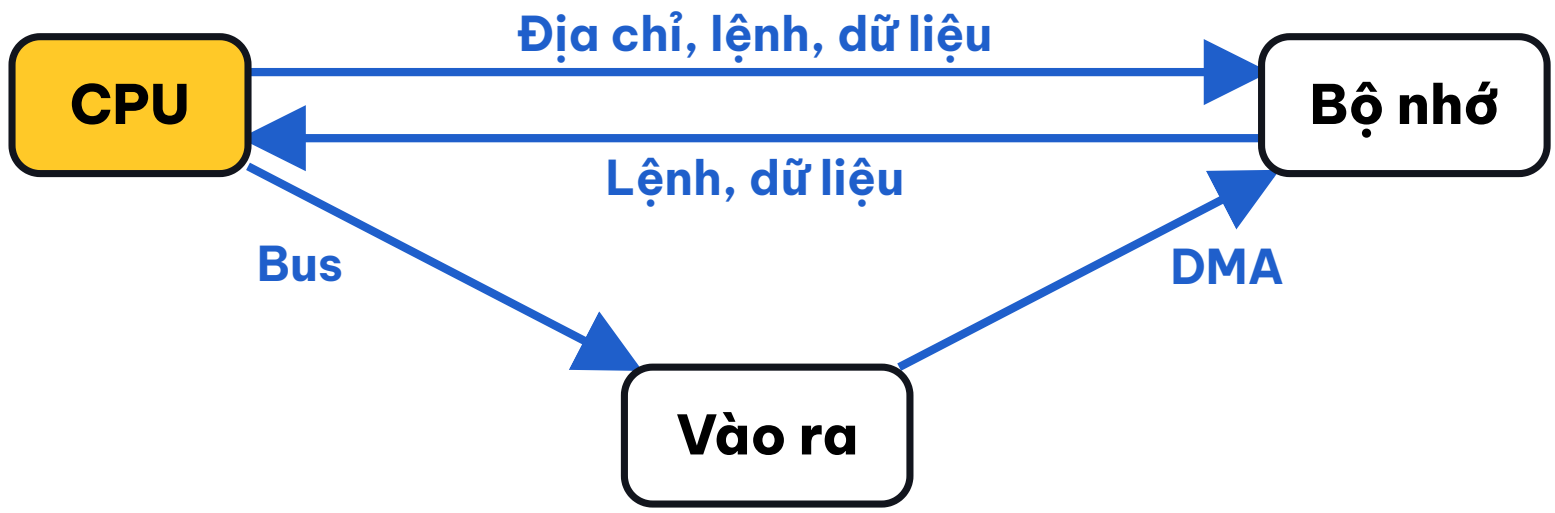


Tầng	Là gì	Ví dụ
ISA	Hợp đồng giữa phần mềm và phần cứng	x86-64, ARM, RISC-V, MIPS
Vi kiến trúc	Cách một chip hiện thực ISA đó	Intel và AMD cùng chạy x86-64

Biết môn này để	Ví dụ
Code nhanh hơn	Duyệt mảng đúng chiều nhanh gấp nhiều lần
Debug sâu hơn	Đọc được crash ở mức mã máy
Tránh lỗi số học	Hiểu tràn số, sai số số thực
Làm nhúng, bảo mật	Đọc hợp ngữ là kỹ năng hằng ngày

2 Mô hình von Neumann

Ý tưởng nền tảng: lệnh và dữ liệu cùng nằm trong bộ nhớ. Muốn đổi chương trình chỉ cần nạp lại bộ nhớ.



Thành phần	Nhiệm vụ
Khối điều khiển	Đọc lệnh, giải mã, phát tín hiệu điều khiển
ALU	Thực hiện phép toán số học, logic
Thanh ghi	Bộ nhớ cực nhanh ngay trong CPU
Bộ nhớ chính	Chứa cả lệnh và dữ liệu
Vào ra	Giao tiếp với bàn phím, ổ đĩa, mạng

VON NEUMANN
Lệnh và dữ liệu chung đường truyền. Đơn giản, nhưng đường truyền thành nút cổ chai.

HARVARD
Tách bộ nhớ lệnh và dữ liệu, đọc song song được. Hay gặp trong vi điều khiển.

CPU hiện đại kết hợp cả hai: bộ nhớ chính chung, nhưng cache L1 tách riêng lệnh và dữ liệu.

3 Một lệnh chạy thế nào?

CPU chỉ lặp mãi một việc: nạp lệnh, giải mã, thực thi. Đoạn code dưới đây là một CPU đầy đủ chu trình đó.

Nạp → Giải-mã → Thực-thi → Ghi-kết-quả

```
mem = [("LOAD", 5), ("ADD", 6), ("STORE", 7), ("HALT", 0),
       0, 20, 22, 0]              # 4 ô đầu là lệnh
pc, acc = 0, 0

while True:
    op, addr = mem[pc]            # nạp lệnh ở địa chỉ PC
    pc += 1
    if op == "LOAD":              # giải mã và thực thi
        acc = mem[addr]
    elif op == "ADD":
        acc += mem[addr]
    elif op == "STORE":
        mem[addr] = acc
    elif op == "HALT":
        break
    print(f"PC={pc} {op:5} ACC={acc}")

print("Ô nhớ 7 =", mem[7])
```

KẾT QUẢ KHI CHẠY

```
PC=1 LOAD  ACC=20
PC=2 ADD   ACC=42
PC=3 STORE ACC=42
Ô nhớ 7 = 42
```

- ✓ PC (program counter) giữ địa chỉ lệnh kế tiếp.
- ✓ Rẽ nhánh chỉ đơn giản là ghi một giá trị mới vào PC.
- ✓ Lệnh và dữ liệu nằm chung một mảng, đúng ý tưởng von Neumann.

4 Tám ý tưởng lớn

Patterson và Hennessy tóm cả ngành trong tám ý tưởng. Chúng lặp lại khắp cuốn cẩm nang này.

Ý tưởng	Ví dụ
Thiết kế theo định luật Moore	Thiết kế cho công nghệ lúc chip ra đời
Trừu tượng hóa	ISA che đi chi tiết bên trong chip
Làm nhanh việc thường gặp	Tối ưu lệnh hay dùng trước
Song song	Nhiều lõi, SIMD
Pipeline	Nhiều lệnh ở các tầng khác nhau
Dự đoán	Đoán nhánh, sai thì sửa
Phân cấp bộ nhớ	Thanh ghi, cache, RAM, đĩa
Dư thừa để tin cậy	RAID, bộ nhớ ECC

Các ý tưởng này dùng được cả ở phần mềm: cache trong web, xử lý song song, đoán trước để nạp sẵn dữ liệu.

Định luật Moore: số transistor trên chip tăng gấp đôi sau khoảng 2 năm. Đây là quan sát kinh tế và kỹ thuật, không phải định luật vật lý.

5 Bức tường công suất

Trước đây chip nhanh hơn nhờ tăng xung nhịp. Khoảng giữa những năm 2000 điều đó dừng lại, vì chip quá nóng.

Công suất động: P ≈ C × V² × f. Trong đó C là điện dung, V là điện áp, f là tần số.

```
def power(v, f):
    return v ** 2 * f                  # so với mức gốc bằng 1

print("Giảm 15% cả điện áp và tần số:", round(power(.85, .85), 2))
print("Hai lõi chạy mức đó:", round(2 * power(.85, .85), 2))
print("Hiệu năng hai lõi:", round(2 * .85, 2))
```

KẾT QUẢ KHI CHẠY

```
Giảm 15% cả điện áp và tần số: 0.61
Hai lõi chạy mức đó: 1.23
Hiệu năng hai lõi: 1.7
```

Giải pháp	Ý nghĩa
Nhiều lõi	Tăng hiệu năng mà không tăng xung nhịp
Lõi hiệu năng, lõi tiết kiệm	Việc nhẹ chạy trên lõi nhỏ
Khối chuyên dụng	GPU, NPU, bộ giải mã video

- ✓ Hai lõi chậm hơn một chút cho hiệu năng 1,7 lần mà chỉ tốn 1,23 lần điện.
- ✓ Phần mềm phải viết song song mới tận dụng được nhiều lõi.
- ✓ Laptop tự giảm xung khi nóng, nên tản nhiệt ảnh hưởng thẳng tới tốc độ.

Con số trên minh họa theo công thức, giả sử hiệu năng tỉ lệ với tần số và công việc chia đều cho hai lõi.

6 Công thức hiệu năng CPU

Công thức quan trọng nhất của môn học. Thời gian chạy phụ thuộc số lệnh, số chu kỳ mỗi lệnh và tần số.

Thời gian CPU = Số lệnh × CPI ÷ Tần số xung nhịp

```python
def cpu_time(instructions, cpi, ghz):
    return instructions * cpi / (ghz * 1e9)

a = cpu_time(1e9, cpi=2.0, ghz=4)
b = cpu_time(1e9, cpi=1.2, ghz=3)
print(f"Máy A 4 GHz: {a:.3f} s")
print(f"Máy B 3 GHz: {b:.3f} s")
print(f"B nhanh hơn A {a / b:.2f} lần")
```

KẾT QUẢ KHI CHẠY

```
Máy A 4 GHz: 0.500 s
Máy B 3 GHz: 0.400 s
B nhanh hơn A 1.25 lần
```

Yếu tố	Do ai quyết định
Số lệnh	Thuật toán, ngôn ngữ, trình biên dịch, ISA
CPI	Vi kiến trúc, cách code dùng cache và nhánh
Chu kỳ xung nhịp	Thiết kế mạch, công nghệ chế tạo

- ✓ Hiệu năng = 1 / thời gian. A nhanh hơn B n lần khi thời gian B / thời gian A = n.
- ✓ Tần số 4 GHz nghĩa là một chu kỳ dài 0,25 ns.
- ✓ Máy GHz thấp hơn vẫn có thể thắng, như ví dụ trên.

7 CPI trung bình

Mỗi loại lệnh tốn số chu kỳ khác nhau. CPI của chương trình là trung bình có trọng số theo tỉ lệ từng loại.

```
mix = {"ALU": (0.50, 1), "Load": (0.20, 5),
       "Store": (0.10, 3), "Branch": (0.20, 2)}

cpi = sum(rate * cycles for rate, cycles in mix.values())
print("CPI =", round(cpi, 2))

for name, (rate, cycles) in mix.items():
    share = rate * cycles / cpi
    print(f"{name:6} chiếm {share:.0%} thời gian")

new = cpi - 0.20 * 5 + 0.20 * 2        # Load chỉ còn 2 chu kỳ
print(f"Tăng tốc Load: CPI {new:.2f}, nhanh {cpi / new:.2f} lần")
```

KẾT QUẢ KHI CHẠY

```
CPI = 2.2
ALU    chiếm 23% thời gian
Load   chiếm 45% thời gian
Store  chiếm 14% thời gian
Branch chiếm 18% thời gian
Tăng tốc Load: CPI 1.60, nhanh 1.38 lần
```

Loại lệnh	Tỉ lệ	CPI	Đóng góp
ALU	50%	1	0,5
Load	20%	5	1,0
Store	10%	3	0,3
Branch	20%	2	0,4

Làm nhanh việc thường gặp: Load chỉ 20% số lệnh nhưng chiếm tới 45% thời gian, nên đáng tối ưu nhất.

8 Định luật Amdahl

Tăng tốc một phần chỉ có ích theo tỉ lệ phần đó chiếm. Ví dụ dưới đây: 90% chương trình chạy song song được.

Tăng tốc = 1 / ((1 - p) + p / s). Trong đó p là tỉ lệ được cải thiện, s là mức tăng tốc của phần đó.

```
def speedup(p, s):
    return 1 / ((1 - p) + p / s)

for cores in (2, 4, 8, 16, 1024):
    print(f"{cores:<5} lõi: nhanh {speedup(.9, cores):5.2f} lần")

print("Trần khi có vô hạn lõi:", round(1 / (1 - .9), 2))
```

KẾT QUẢ KHI CHẠY

```
2     lõi: nhanh  1.82 lần
4     lõi: nhanh  3.08 lần
8     lõi: nhanh  4.71 lần
16    lõi: nhanh  6.40 lần
1024  lõi: nhanh  9.91 lần
Trần khi có vô hạn lõi: 10.0
```

Bài học	Áp dụng
Đo trước khi tối ưu	Tìm phần chiếm nhiều thời gian nhất
Phần tuần tự là trần	Giảm phần phải chờ, phải khóa
Đừng tối ưu chỗ 1%	Nhanh gấp 100 lần vẫn gần như không đổi

Bài thi hay hỏi ngược: muốn nhanh tổng thể 2 lần thì phần chiếm 80% phải nhanh bao nhiêu lần. Đáp số là 2,67 lần.

9 Đọc thông số CPU

GHz không nói lên tất cả. Đây là CPU của máy dùng để chạy các ví dụ trong cẩm nang.

Thông số	Intel Core i5-1035G1	Ý nghĩa
Lõi, luồng	4 lõi, 8 luồng	Việc chạy song song thật
Cache L1	48 KB dữ liệu mỗi lõi	Nhanh nhất
Cache L2	512 KB mỗi lõi	Chậm hơn L1
Cache L3	6 MB dùng chung	Tầng cuối trước RAM

Chỉ số	Vì sao dễ gây hiểu lầm
GHz	Khác kiến trúc thì không so được, xung cao chỉ giữ một lúc
MIPS (triệu lệnh/giây)	Một lệnh của mỗi ISA làm được lượng việc khác nhau
Số lõi	Việc tuần tự chỉ dùng một lõi
Điểm benchmark	Chỉ đúng với loại việc được đo

Cách so đúng nhất: đo chính công việc của bạn. Không có điều kiện thì xem benchmark gần với việc đó, như biên dịch code, dựng video.

```
wmic cpu get name,numberofcores   # Windows
lscpu                             # Linux
```

10 Nhị phân và thập lục phân

Máy tính chỉ có bit. Hex là cách viết gọn: mỗi chữ số hex ứng với đúng 4 bit, nên đọc mã máy rất tiện.

```python
def to_base(n, base):
    digits = "0123456789ABCDEF"
    out = ""
    while n:
        n, r = divmod(n, base)      # chia liên tiếp, lấy phần dư
        out = digits[r] + out
    return out or "0"

print(to_base(2026, 2), to_base(2026, 16))
print(bin(2026), hex(2026), int("7EA", 16))
```

KẾT QUẢ KHI CHẠY

```
11111101010 7EA
0b11111101010 0x7ea 2026
```

Hex	Nhị phân	Hex	Nhị phân
0	0000	**8**	1000
3	0011	**A**	1010
5	0101	**C**	1100
7	0111	**F**	1111

Tiền tố	Giá trị	Hay gặp
KiB, MiB, GiB	2^{10}, 2^{20}, 2^{30}	RAM, cache
KB, MB, GB	10^3, 10^6, 10^9	Ổ đĩa, tốc độ mạng

Ổ 1 TB hiện 931 GB trên Windows không phải bị thiếu: 10^{12} byte chia cho 2^{30} được khoảng 931.

11 Số nguyên có dấu

Số bù 2 là cách máy tính dùng để biểu diễn số âm. Lấy đảo bit rồi cộng 1. Mạch cộng dùng chung cho cả số âm.

```
def twos(x, bits=8):
    return format(x & ((1 << bits) - 1), f"0{bits}b")

def value(text):
    n = int(text, 2)
    return n - (1 << len(text)) if text[0] == "1" else n

for x in (5, -5, 127, -128):
    print(f"{x:<5} = {twos(x)}")
print(value("11111011"))
```

KẾT QUẢ KHI CHẠY

```
5     = 00000101
-5    = 11111011
127   = 01111111
-128  = 10000000
-5
```

4 bit	Không dấu	Dấu lượng	Bù 1	Bù 2
0111	7	7	7	7
1000	8	-0	-7	-8
1111	15	-7	-0	-1

Số bit n	Không dấu	Bù 2
8	0 đến 255	-128 đến 127
32	0 đến 4.294.967.295	-2.147.483.648 đến 2.147.483.647

Bù 2 chỉ có một số 0, và phép trừ a - b chỉ là a + (-b). Vì vậy mọi CPU hiện đại đều dùng nó.

12 Tràn số

Số bit có hạn. Kết quả vượt phạm vi thì bị cắt, và cộng hai số dương có thể ra số âm.

```c
#include <stdio.h>
int main(void) {
    unsigned char a = 200, b = 100;
    signed char c = 100, d = 100;
    printf("%d\n", (unsigned char)(a + b));
    printf("%d\n", (signed char)(c + d));
    return 0;
}
```

KẾT QUẢ BIÊN DỊCH VÀ CHẠY

```
44
-56
```

```python
def add8(a, b):
    raw = (a + b) & 0xFF
    r = raw - 256 if raw >= 128 else raw
    overflow = (a < 0) == (b < 0) and (r < 0) != (a < 0)
    return r, overflow

print(add8(100, 100), add8(-100, -100), add8(100, -100))
```

KẾT QUẢ KHI CHẠY

```
(-56, True) (56, True) (0, False)
```

Phép cộng bù 2	Có tràn không
Dương + dương ra âm	Tràn
Âm + âm ra dương	Tràn
Dương + âm	Không bao giờ tràn

Cờ carry báo tràn số không dấu, cờ overflow báo tràn số có dấu. Trong C, tràn số có dấu là hành vi không xác định.

13 Tràn số ngoài đời thật

Lỗi tưởng như nhỏ này từng làm nổ tên lửa và làm phải sửa lại bộ đếm của YouTube.

Sự cố	Chuyện gì xảy ra
Ariane 5, 1996	Số thực 64 bit đổi sang số nguyên 16 bit bị tràn, tên lửa nổ khoảng 40 giây sau khi phóng
YouTube, 2014	Lượt xem Gangnam Style vượt 2.147.483.647, phải đổi bộ đếm sang 64 bit
Năm 2038	Thời gian Unix 32 bit có dấu sẽ tràn

```python
from datetime import datetime, timezone

limit = 2 ** 31 - 1
print(limit)
print(datetime.fromtimestamp(limit, timezone.utc))
```

KẾT QUẢ KHI CHẠY

```
2147483647
2038-01-19 03:14:07+00:00
```

- ✓ Tính tổng tiền, số lượt xem nên dùng số 64 bit ngay từ đầu.
- ✓ Tính trung điểm tìm kiếm nhị phân bằng lo + (hi - lo) / 2 để tránh tràn.
- ✓ Nhân hai số int trong C rồi mới gán cho long long thì vẫn tràn.
- ✓ Kiểm tra giới hạn khi đổi kiểu từ lớn xuống nhỏ.

14 Phép toán bit

Bật, tắt, kiểm tra từng bit. Gặp ở driver, nén dữ liệu, cờ quyền, mạng và rất nhiều câu phỏng vấn.

```
READ, WRITE, EXEC = 0b100, 0b010, 0b001

perm = READ | WRITE                # bật bit
print(format(perm, "03b"))
print(bool(perm & WRITE))          # kiểm tra bit
perm &= ~WRITE                     # tắt bit
perm ^= EXEC                       # đảo bit
print(format(perm, "03b"))

x = 0b10110100
print(x >> 2 & 0b111)              # lấy bit 2 đến 4
print(x & (x - 1) == 0, 64 & 63 == 0)   # có phải lũy thừa của 2?
```

KẾT QUẢ KHI CHẠY

```
110
True
101
5
False True
```

Việc	Biểu thức
Bật bit k	`x | (1 << k)`
Tắt bit k	`x & ~(1 << k)`
Đảo bit k	`x ^ (1 << k)`
Kiểm tra bit k	`(x >> k) & 1`
Nhân, chia cho 2^k	`x << k, x >> k`

Dịch logic điền 0, dùng cho số không dấu. Dịch số học giữ bit dấu: -8 dịch phải 1 bit ra -4.

15 Thứ tự byte

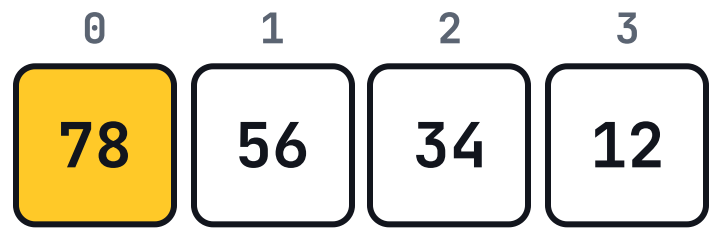
Số nhiều byte lưu trong bộ nhớ theo thứ tự nào? Little endian đặt byte thấp trước, big endian đặt byte cao trước.

```
import struct, sys

x = 0x12345678
print("Little:", struct.pack("<I", x).hex(" "))
print("Big:   ", struct.pack(">I", x).hex(" "))
print("Máy này:", sys.byteorder)
```

KẾT QUẢ KHI CHẠY

```
Little: 78 56 34 12
Big:    12 34 56 78
Máy này: little
```



Hệ thống	Thứ tự
x86-64	Little endian
ARM, RISC-V	Thường little endian
MARS trên PC	Little endian
Giao thức mạng	Big endian, nên gọi là network byte order

- ✓ Đọc file nhị phân, gói tin mạng là lúc gặp lỗi thứ tự byte.
- ✓ Dùng htonl, ntohl trong C để đổi giữa thứ tự máy và thứ tự mạng.
- ✓ Nhìn bộ nhớ trong debugger trên x86 sẽ thấy số bị ngược từng byte.

16 Ký tự và tiếng Việt

Chữ cũng chỉ là số. Tiếng Việt chiếm nhiều byte hơn, và một chữ có thể có hai cách lưu khác nhau.

```python
import unicodedata as ud

for ch in "Aêệ€":
    b = ch.encode()
    print(ch, f"U+{ord(ch):04X}", len(b), "byte:", b.hex(" "))

a = "Việt"
b = ud.normalize("NFD", a)        # dấu tách riêng
print(a == b, len(a), len(b))
print(a == ud.normalize("NFC", b))
```

KẾT QUẢ KHI CHẠY

```
A U+0041 1 byte: 41
ê U+00EA 2 byte: c3 aa
ệ U+1EC7 3 byte: e1 bb 87
€ U+20AC 3 byte: e2 82 ac
False 4 6
True
```

Bảng mã	Đặc điểm
ASCII	7 bit, 128 ký tự, không có tiếng Việt
Unicode	Đánh số cho mọi ký tự, chưa nói cách lưu
UTF-8	1 đến 4 byte mỗi ký tự, tương thích ASCII

Lỗi thật: tìm kiếm tên người Việt không ra, vì dữ liệu lưu NFC mà từ khóa gõ từ Mac là NFD. Hãy chuẩn hóa về NFC trước khi so sánh.

Mẹo nhớ bảng ASCII: '0' là 48, 'A' là 65, 'a' là 97. Chữ hoa và chữ thường cách nhau đúng 32, tức một bit.

17 Số thực IEEE 754

Số thực lưu như ký hiệu khoa học nhị phân: dấu, số mũ và phần định trị. Giá trị = (-1)ˢ × 1,f × 2^(e - bias).




```python
import struct

def parts(x):
    bits = struct.unpack(">I", struct.pack(">f", x))[0]
    s, e, f = bits >> 31, bits >> 23 & 255, bits & (2 ** 23 - 1)
    print(f"{x}: {s} {e:08b} {f:023b}")
    return (-1) ** s * (1 + f / 2 ** 23) * 2.0 ** (e - 127)

print(parts(-6.75))
print(parts(0.15625))
```

KẾT QUẢ KHI CHẠY

```
-6.75: 1 10000001 10110000000000000000000
-6.75
0.15625: 0 01111100 01000000000000000000000
0.15625
```

Định dạng	Dấu	Mũ	Định trị	Bias	Độ chính xác
float (32 bit)	1	8	23	127	Khoảng 7 chữ số
double (64 bit)	1	11	52	1023	Khoảng 16 chữ số

- ✓ Có số 1 ẩn trước dấu phẩy, nên 23 bit lưu được 24 bit độ chính xác.
- ✓ Số mũ cộng thêm bias để không phải lưu số âm.
- ✓ AI còn dùng các định dạng ngắn hơn như FP16, BF16 để tính nhanh.

18 Đổi số thực sang IEEE 754

Dạng bài chắc chắn có. Ví dụ đổi -6,75 sang float 32 bit theo bốn bước.

1 Xét dấu
Số âm nên S = 1.

2 Đổi phần nguyên, phần thập phân
6 = 110. Phần 0,75: nhân 2 lấy phần nguyên, được 0,11. Vậy 6,75 = 110,11.

3 Chuẩn hóa
110,11 = 1,1011 × 2^2. Phần định trị là 1011, thêm số 0 cho đủ 23 bit.

4 Tính số mũ
e = 2 + 127 = 129 = 10000001.

1	10000001	10110000000000000000000

```
import struct

bits = struct.unpack(">I", struct.pack(">f", -6.75))[0]
print(f"{bits:032b}")
print(f"Hex: {bits:08X}")
```

KẾT QUẢ KHI CHẠY

```
11000000110110000000000000000000
Hex: C0D80000
```

0,1 trong nhị phân là số vô hạn tuần hoàn 0,000110011..., giống 1/3 trong thập phân. Đó là gốc của lỗi ở trang sau.

19 Vì sao 0.1 + 0.2 khác 0.3?

0,1 không biểu diễn chính xác được bằng nhị phân. Máy tính lưu giá trị gần nhất, nên phép tính có sai số nhỏ.

```
from decimal import Decimal
import math

print(0.1 + 0.2)
print(Decimal(0.1))
print(0.1 + 0.2 == 0.3, math.isclose(0.1 + 0.2, 0.3))
print(Decimal("0.1") + Decimal("0.2"))

price = 19999                          # lưu tiền bằng số nguyên đồng
print(price * 3)
```

KẾT QUẢ KHI CHẠY

```
0.30000000000000004
0.1000000000000000055511151231257827021181583404541015625
False True
0.3
59997
```

Quy tắc	Cách làm
So sánh số thực	Dùng sai số cho phép, không dùng ==
Tiền	Lưu số nguyên hoặc kiểu Decimal, NUMERIC
Vòng lặp	Đếm bằng số nguyên, đừng cộng dồn 0,1

Năm 1991, hệ thống Patriot ở Dhahran chạy liên tục khoảng 100 giờ. Sai số khi lưu 0,1 giây tích lũy thành 0,34 giây, nên không đánh chặn được tên lửa Scud.

Python hiển thị 0.1 gọn vì nó in số ngắn nhất khôi phục được đúng giá trị đó. Giá trị lưu thật là dòng thứ hai.

20 Đổi thứ tự cộng, đổi kết quả

Phép cộng số thực không có tính kết hợp. Số quá nhỏ cộng với số quá lớn sẽ biến mất.

```
import math, struct

a = 1e16
print(a + 1 - a, a - a + 1)        # đổi thứ tự, đổi kết quả

total = 0.0
for v in (1e16, 1.0, -1e16):
    total += v
print(total, math.fsum((1e16, 1.0, -1e16)))

def f32(x):
    return struct.unpack("f", struct.pack("f", x))[0]

print(f32(16777216.0), f32(16777217.0))   # 2 mũ 24 và 2 mũ 24 + 1
```

KẾT QUẢ KHI CHẠY

```
0.0 1.0
0.0 1.0
16777216.0 16777216.0
```

Tình huống	Vì sao lỗi	Cách tránh
Cộng số lớn với số bé	Số bé bị làm tròn mất	Cộng các số bé trước, dùng fsum
Lưu ID lớn trong float	float chỉ đúng tới 2^{24}	Dùng số nguyên
JSON số lớn trong JavaScript	Số JS là double, đúng tới 2^{53}	Truyền ID dạng chuỗi

21 Giá trị đặc biệt của số thực

Một số mẫu bit được dành riêng. Hiểu chúng để không bị bất ngờ khi chia cho 0 hay gặp NaN.

Mũ e	Định trị f	Ý nghĩa
0	0	Số 0, có cả +0 và −0
0	Khác 0	Số dưới chuẩn, rất gần 0
1 đến 254	Bất kỳ	Số thường
255	0	Vô cực
255	Khác 0	NaN, không phải số

```python
import math

inf, nan = float("inf"), float("nan")
print(inf + 1, inf - inf, 1 / inf)
print(nan == nan, math.isnan(nan))
print(0.0 == -0.0, math.copysign(1, -0.0))
print(5e-324, 5e-324 / 2)          # số dương nhỏ nhất của double
```

KẾT QUẢ KHI CHẠY

```
inf nan 0.0
False True
True -1.0
5e-324 0.0
```

- NaN không bằng chính nó, nên phải dùng isnan để kiểm tra.
- Một giá trị NaN sẽ lan khắp phép tính, hay gặp khi huấn luyện mô hình AI.
- Chia số thực cho 0 ở C ra vô cực, chia số nguyên cho 0 thì chương trình crash.

22 Cổng logic

Mọi thứ trong CPU dựng từ vài loại cổng. Chỉ riêng cổng NAND đã đủ để tạo ra tất cả các cổng khác.

A	B	AND	OR	XOR	NAND	NOR
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

```python
def nand(a, b): return 1 - (a & b)
def not_(a):    return nand(a, a)
def and_(a, b): return not_(nand(a, b))
def or_(a, b):  return nand(not_(a), not_(b))
def xor(a, b):
    n = nand(a, b)
    return nand(nand(a, n), nand(b, n))

for a in (0, 1):
    for b in (0, 1):
        print(a, b, "AND", and_(a, b), "OR", or_(a, b),
              "XOR", xor(a, b))
```

KẾT QUẢ KHI CHẠY

```
0 0 AND 0 OR 0 XOR 0
0 1 AND 0 OR 1 XOR 1
1 0 AND 0 OR 1 XOR 1
1 1 AND 1 OR 1 XOR 0
```

Transistor chỉ là công tắc điều khiển bằng điện. Vài transistor ghép lại thành một cổng, CPU hiện đại có hàng tỉ transistor.

23 Đại số Boole

Rút gọn biểu thức nghĩa là bớt cổng, chip nhỏ hơn và nhanh hơn. Máy có thể kiểm tra mọi trường hợp để xác nhận.

Luật	Công thức
De Morgan	¬(A · B) = ¬A + ¬B, ¬(A + B) = ¬A · ¬B
Phân phối	A · (B + C) = A·B + A·C
Hấp thụ	A + A·B = A
Phần bù	A + ¬A = 1, A · ¬A = 0

```
from itertools import product

def same(f, g, n):
    return all(f(*v) == g(*v) for v in product((0, 1), repeat=n))

print(same(lambda a, b: 1 - (a & b),
           lambda a, b: (1 - a) | (1 - b), 2))        # De Morgan
print(same(lambda a, b: a | (a & b), lambda a, b: a, 2))
# F = A'B + AB + AB' rút gọn thành A + B
print(same(lambda a, b: ((1 - a) & b) | (a & b) | (a & (1 - b)),
           lambda a, b: a | b, 2))
```

KẾT QUẢ KHI CHẠY

```
True
True
True
```

Bìa Karnaugh: xếp bảng chân trị sao cho ô kề nhau chỉ khác một bit, rồi khoanh nhóm 2, 4, 8 ô số 1 để rút gọn. Hàm trên có ba ô bằng 1, khoanh hai nhóm được A + B.

24 Mux và decoder

Hai khối có mặt khắp đường dữ liệu. Mux chọn một trong nhiều đầu vào. Decoder bật đúng một đường ra.

```
def mux(sel, *inputs):
    return inputs[sel]

def decoder(n, bits=2):
    return [int(i == n) for i in range(2 ** bits)]

print(mux(0, "thanh ghi", "hằng số"))    # ALUSrc của MIPS
print(mux(1, "thanh ghi", "hằng số"))
print(decoder(2))
```

KẾT QUẢ KHI CHẠY

```
thanh ghi
hằng số
[0, 0, 1, 0]
```

Khối	Công dụng trong CPU
Mux	Chọn đầu vào ALU, chọn PC + 4 hay địa chỉ nhánh
Decoder	Chọn thanh ghi, chọn hàng trong bộ nhớ, giải mã opcode
Encoder	Ngược decoder, dùng cho bộ ưu tiên ngắt

Trong code, if else ứng với mux: mạch tính cả hai nhánh rồi chọn một. Tính cả hai cùng lúc nhanh hơn chờ quyết định.

- ✓ Mux 2 đường vào cần 1 bit chọn, mux 2^n đường vào cần n bit chọn.
- ✓ Decoder n bit có 2^n đường ra.
- ✓ Decoder 5 bit chọn 1 trong 32 thanh ghi của MIPS.

25 Bộ cộng

Cộng 1 bit dùng hai cổng XOR, hai cổng AND, một cổng OR. Nối 32 khối lại là bộ cộng 32 bit.

```
def full_adder(a, b, c):
    s = a ^ b ^ c
    carry = (a & b) | (c & (a ^ b))
    return s, carry

def ripple(x, y, bits=8, sub=0):
    carry, out = sub, 0                    # trừ: đảo y và cộng thêm 1
    for i in range(bits):
        b = (y >> i & 1) ^ sub
        s, carry = full_adder(x >> i & 1, b, carry)
        out |= s << i
    return out, carry

print(ripple(100, 55))
print(ripple(200, 100))                    # tràn 8 bit, nhớ bằng 1
print(ripple(100, 55, sub=1))
```

KẾT QUẢ KHI CHẠY

```
(155, 0)
(44, 1)
(45, 1)
```

Loại	Đặc điểm
Ripple carry	Đơn giản, nhưng bit nhớ phải chạy qua cả 32 khối
Carry lookahead	Tính trước bit nhớ, nhanh hơn, tốn nhiều cổng hơn

Một mạch dùng chung cho cộng và trừ: a − b = a + ~b + 1. Tín hiệu trừ vừa đảo b, vừa làm bit nhớ vào.

26 ALU

ALU gom các phép tính vào một khối. Tín hiệu điều khiển chọn phép nào, cờ Zero dùng cho lệnh rẽ nhánh.

```
MASK = 0xFFFFFFFF

def signed(x):
    return x - (1 << 32) if x >> 31 else x

def alu(ctrl, a, b):
    result = {0b0000: a & b,
              0b0001: a | b,
              0b0010: (a + b) & MASK,
              0b0110: (a - b) & MASK,
              0b0111: int(signed(a) < signed(b)),
              0b1100: ~(a | b) & MASK}[ctrl]
    return result, int(result == 0)

print(alu(0b0010, 7, 5), alu(0b0110, 5, 5))
print(alu(0b0111, MASK, 1))           # -1 < 1
```

KẾT QUẢ KHI CHẠY

```
(12, 0) (0, 1)
(1, 0)
```

Mã điều khiển	Phép toán	Dùng cho
0000	AND	and, andi
0010	Cộng	add, lw, sw
0110	Trừ	sub, beq
0111	Nhỏ hơn thì 1	slt
1100	NOR	nor

beq dùng ALU để trừ hai thanh ghi rồi xem cờ Zero.

27 Phép nhân nhị phân

Nhân tay trong hệ 2 rất đơn giản: gặp bit 1 thì cộng số bị nhân đã dịch. Phần cứng làm đúng như vậy.

```python
def multiply(a, b, bits=4):
    product = 0
    for i in range(bits):
        if b & 1:
            product += a                # bit thấp bằng 1
        print(f"{i + 1}: b={b:04b} a={a:08b} tích={product:08b}")
        a <<= 1                         # số bị nhân dịch trái
        b >>= 1                         # số nhân dịch phải
    return product

print(multiply(0b0110, 0b0101))
```

KẾT QUẢ KHI CHẠY

```
1: b=0101 a=00000110 tích=00000110
2: b=0010 a=00001100 tích=00000110
3: b=0001 a=00011000 tích=00011110
4: b=0000 a=00110000 tích=00011110
30
```

- Nhân hai số n bit cho kết quả tới 2n bit, nên MIPS lưu vào cặp HI, LO.
- Phần cứng thật dùng nhiều bộ cộng song song, nhanh hơn hẳn cách này.
- Nhân với 2^k chỉ là dịch trái, trình biên dịch luôn tận dụng điều đó.

Thuật toán Booth	Ý tưởng
Mục đích	Nhân số có dấu, ít phép cộng hơn khi có chuỗi bit 1
Quy tắc 2 bit	10: trừ, 01: cộng, 00 và 11: chỉ dịch

Trang sau là ví dụ Booth từng bước. Một số trường có ra dạng bài này.

28 Thuật toán Booth

Xét cặp bit (Q0, Q-1). Sau mỗi bước, dịch phải số học cả ba thanh ghi A, Q, Q-1.

```
def booth(m, q, bits=4):
    mask = (1 << bits) - 1
    a, q, q1, m = 0, q & mask, 0, m & mask
    print(f"0: A={a:0{bits}b} Q={q:0{bits}b} Q-1={q1}")
    for step in range(bits):
        pair = (q & 1, q1)
        if pair == (1, 0):
            a = (a - m) & mask              # A = A - M
        elif pair == (0, 1):
            a = (a + m) & mask              # A = A + M
        q1 = q & 1                          # dịch phải số học
        q = (q >> 1) | (a & 1) << (bits - 1)
        a = (a >> 1) | (a & (1 << (bits - 1)))
        print(f"{step + 1}: A={a:0{bits}b} Q={q:0{bits}b} Q-1={q1}")
    r = a << bits | q
    return r - (1 << 2 * bits) if r >> (2 * bits - 1) else r

print("Kết quả:", booth(3, -4))
```

KẾT QUẢ KHI CHẠY

```
0: A=0000 Q=1100 Q-1=0
1: A=0000 Q=0110 Q-1=0
2: A=0000 Q=0011 Q-1=0
3: A=1110 Q=1001 Q-1=1
4: A=1111 Q=0100 Q-1=1
Kết quả: -12
```

Khởi tạo A = 0, Q là số nhân, Q-1 = 0, M là số bị nhân. Lặp n lần: xét cặp, cộng hoặc trừ M vào A, rồi dịch. Ghép A và Q được kết quả 2n bit.

Lưu ý: số bị nhân nhỏ nhất, như -8 với 4 bit, làm A bị tràn. Trường hợp này cần thêm 1 bit cho A.

29 Phép chia nhị phân

Chia như chia tay: thử trừ, được thì ghi 1, không được thì khôi phục và ghi 0. MIPS lưu thương ở LO, dư ở HI.

```
def divide(n, d, bits=4):
    r, q = 0, n
    for i in range(bits):
        r = r << 1 | q >> (bits - 1) & 1      # dịch trái cặp R, Q
        q = q << 1 & ((1 << bits) - 1)
        if r >= d:
            r -= d
            q |= 1                            # trừ được: ghi 1
        print(f"Bước {i + 1}: dư={r:04b} thương={q:04b}")
    return q, r

print(divide(0b1101, 0b0011))
print(divmod(13, 3))
```

KẾT QUẢ KHI CHẠY

```
Bước 1: dư=0001 thương=1010
Bước 2: dư=0000 thương=0101
Bước 3: dư=0000 thương=1010
Bước 4: dư=0001 thương=0100
(4, 1)
(4, 1)
```

Trường hợp	Kết quả trong C	Trong Python
7 / 2	3	3 với //
-7 / 2	-3, làm tròn về 0	-4 với //, làm tròn xuống
-7 % 2	-1	1

Phép chia chậm hơn phép cộng rất nhiều. Vì vậy trình biên dịch đổi chia cho hằng số thành nhân và dịch, xem ở phần 5.

30 Flip-flop, thanh ghi, xung nhịp

Cổng logic không nhớ gì. Flip-flop giữ được 1 bit, ghép lại thành thanh ghi. Xung nhịp đồng bộ tất cả.

```
class DFlipFlop:
    q = 0
    def tick(self, d):              # chỉ đổi ở sườn xung nhịp
        self.q = d

reg = [DFlipFlop() for _ in range(8)]      # thanh ghi 8 bit
for value in (5, 200):
    for i, f in enumerate(reg):
        f.tick(value >> i & 1)
    print("Sau xung nhịp:", sum(f.q << i for i, f in enumerate(reg)))
```

KẾT QUẢ KHI CHẠY

```
Sau xung nhịp: 5
Sau xung nhịp: 200
```

Loại mạch	Đặc điểm	Ví dụ
Tổ hợp	Đầu ra chỉ phụ thuộc đầu vào hiện tại	ALU, mux
Tuần tự	Có trạng thái, cần xung nhịp	Thanh ghi, PC

Chu kỳ xung nhịp phải đủ dài cho đường đi chậm nhất giữa hai thanh ghi. Đó là đường găng, và nó quyết định tần số tối đa.

31 Tập lệnh, RISC và CISC

Tập lệnh là ngôn ngữ mà CPU hiểu. MIPS đơn giản, rất hợp để học, nên nhiều giáo trình kiến trúc máy tính dùng nó.

Tiêu chí	RISC	CISC
Ví dụ	MIPS, ARM, RISC-V	x86-64
Độ dài lệnh	Cố định, thường 4 byte	Thay đổi, 1 đến 15 byte
Truy cập bộ nhớ	Chỉ qua load, store	Lệnh tính toán cũng được
Số thanh ghi	Nhiều, 32	Ít hơn, 16
Giải mã	Đơn giản	Phức tạp

Ranh giới ngày nay đã mờ: CPU x86 tách lệnh phức tạp thành các vi lệnh kiểu RISC ở bên trong.

Tập lệnh	Dùng ở đâu
x86-64	Máy tính, server
ARM	Điện thoại, máy Apple, nhiều server cloud
RISC-V	Tập lệnh mở, đang lên ở nhúng và nghiên cứu
MIPS	Chủ yếu trong giảng dạy và một số thiết bị mạng

Học MIPS rồi sang ARM, RISC-V rất nhanh, vì tư duy giống nhau.

32 Thanh ghi MIPS

MIPS có 32 thanh ghi 32 bit. Số hiệu là phần cứng, còn tên gọi là quy ước để các hàm làm việc được với nhau.

Tên	Số	Công dụng	Hàm con phải giữ?
$zero	0	Luôn bằng 0	Không đổi được
$at	1	Dành cho trình hợp dịch	Không dùng
$v0, $v1	2, 3	Giá trị trả về, mã syscall	Không
$a0 đến $a3	4 đến 7	Tham số hàm	Không
$t0 đến $t7	8 đến 15	Biến tạm	Không
$s0 đến $s7	16 đến 23	Biến cần giữ lâu	Có
$t8, $t9	24, 25	Biến tạm	Không
$k0, $k1	26, 27	Dành cho kernel	Không dùng
$gp	28	Con trỏ toàn cục	Có
$sp	29	Con trỏ stack	Có
$fp	30	Con trỏ khung	Có
$ra	31	Địa chỉ trả về	Có

CÁCH NHỚ
t là temporary, dùng rồi bỏ. s là saved, phải trả lại nguyên vẹn.

NGOÀI RA
Có PC, cặp HI, LO cho nhân chia. Số thực dùng $f0 đến $f31.

33 Chương trình MIPS đầu tiên

Một chương trình có phần .data chứa dữ liệu và phần .text chứa lệnh. Vào ra đi qua syscall, mã dịch vụ đặt trong $v0.

```
        .data
msg:    .asciiz "Xin chao MIPS\n"

        .text
main:   li   $v0, 4           # dịch vụ 4: in chuỗi
        la   $a0, msg         # $a0 = địa chỉ chuỗi
        syscall
        li   $v0, 1           # dịch vụ 1: in số nguyên
        li   $a0, 2026
        syscall
        li   $v0, 10          # dịch vụ 10: kết thúc
        syscall
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
Xin chao MIPS
2026
```

$v0	Dịch vụ	Tham số, kết quả
1	In số nguyên	$a0 = số
4	In chuỗi	$a0 = địa chỉ
5	Đọc số nguyên	Kết quả ở $v0
10	Kết thúc	
11	In ký tự	$a0 = ký tự

Bảng theo tài liệu của MARS. Quên syscall 10 thì chương trình chạy tiếp vào vùng nhớ phía sau.

34 Lệnh số học

Mỗi lệnh làm một việc, tối đa ba toán hạng. Biểu thức dài phải tách thành nhiều lệnh.

```
main:   li   $s0, 17
        li   $s1, 5
        add  $t0, $s0, $s1    # 17 + 5
        sub  $t1, $s0, $s1    # 17 - 5
        addi $t2, $s0, 100    # cộng hằng số
        mul  $t3, $s0, $s1    # chỉ lấy 32 bit thấp
        div  $s0, $s1         # thương vào LO, dư vào HI
        mflo $t4
        mfhi $t5
        li   $v0, 10
        syscall
# Xem: $t0 $t1 $t2 $t3 $t4 $t5
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
$t0=22  $t1=12  $t2=117  $t3=85  $t4=3  $t5=2
```

Lệnh	Ý nghĩa	Lưu ý
add, sub	Cộng, trừ thanh ghi	Báo lỗi khi tràn
addu, subu	Cộng, trừ không báo lỗi	Trình biên dịch C hay dùng
addi	Cộng hằng 16 bit có dấu	Không có subi thật
mult, div	Kết quả vào HI, LO	Đọc bằng mfhi, mflo

C: f = (g + h) - (i + j) cần ba lệnh: add $t0, $s1, $s2, rồi add $t1, $s3, $s4, rồi sub $s0, $t0, $t1.

Dòng # Xem là chú thích để mips.py in thanh ghi. Trong MARS bạn nhìn bảng Registers ở bên phải.

35 Lệnh logic và dịch bit

Dùng để xử lý từng bit, và thay phép nhân, chia cho lũy thừa của 2.

```
main:   li   $t0, 0xF0
        li   $t1, 0x3C
        and  $s0, $t0, $t1
        or   $s1, $t0, $t1
        xor  $s2, $t0, $t1
        sll  $s3, $t0, 2      # dịch trái 2: nhân 4
        srl  $s4, $t0, 4      # dịch phải logic
        li   $t2, -16
        sra  $s5, $t2, 2      # dịch phải số học: chia 4
        srl  $s6, $t2, 28     # dịch logic với số âm
        li   $v0, 10
        syscall
# Xem: $s0 $s1 $s2 $s3 $s4 $s5 $s6
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
$s0=48   $s1=252   $s2=204   $s3=960   $s4=15   $s5=-4   $s6=15
```

Lệnh	Ý nghĩa
and, or, xor, nor	Phép logic trên 32 bit
andi, ori, xori	Hằng 16 bit, mở rộng bằng số 0
sll, srl	Dịch logic, điền 0
sra	Dịch số học, giữ dấu
sllv, srlv, srav	Số bit dịch nằm trong thanh ghi

- ✓ MIPS không có lệnh not thật: not $t0, $t1 là nor $t0, $t1, $zero.
- ✓ Lấy 8 bit thấp: andi $t0, $t1, 0xFF.
- ✓ Không có lệnh move thật: nó là addu với $zero.

36 Nạp và lưu bộ nhớ

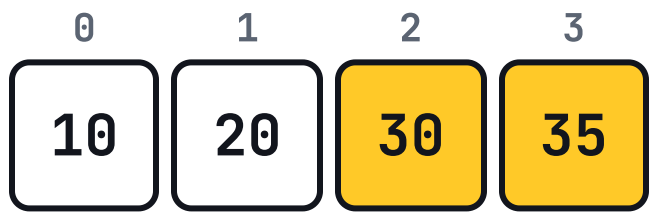
Chỉ lw, sw mới chạm bộ nhớ. Địa chỉ = thanh ghi nền + độ dời. Bộ nhớ đánh địa chỉ theo byte, nên phần tử thứ i của mảng word nằm ở i × 4.

```
        .data
a:      .word 10, 20, 30, 40
        .text
main:   la   $s0, a           # $s0 = địa chỉ a[0]
        lw   $t0, 8($s0)      # a[2]: độ dời 2 × 4
        addi $t0, $t0, 5
        sw   $t0, 12($s0)     # a[3] = a[2] + 5

        li   $s1, 3           # i = 3
        sll  $t1, $s1, 2      # i × 4
        add  $t1, $t1, $s0    # địa chỉ a[i]
        lw   $t2, 0($t1)
        li   $v0, 10
        syscall
# Xem: $t0 $t2
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
$t0=35   $t2=35
```



Lệnh	Kích thước	Lưu ý
lw, sw	4 byte	Địa chỉ phải chia hết cho 4
lh, lhu, sh	2 byte	lh mở rộng dấu, lhu mở rộng 0
lb, lbu, sb	1 byte	Dùng cho chuỗi

Quên nhân 4 là lỗi phổ biến nhất của người mới. Hình trên là mảng sau khi chạy.

37 Rẽ nhánh: if, else

MIPS chỉ có beq, bne so sánh bằng. Muốn so lớn nhỏ thì dùng slt trước. Thường đảo điều kiện để nhảy qua nhánh if.

```
// C: lấy số lớn hơn
if (a > b)
    m = a;
else
    m = b;
```

```
# $s0 = a, $s1 = b, $s2 = m
    slt  $t0, $s1, $s0
    beq  $t0, $zero, else
    move $s2, $s0
    j    end
else: move $s2, $s1
end:
```

```
main:   li   $s0, 7
        li   $s1, 12
        slt  $t0, $s1, $s0   # $t0 = 1 nếu b < a
        beq  $t0, $zero, else
        move $s2, $s0
        j    end
else:   move $s2, $s1
end:    li   $v0, 10
        syscall
# Xem: $s2
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
$s2=12
```

Lệnh	Nhảy khi	Loại
beq, bne	Bằng, khác	Lệnh thật
slt, slti	Không nhảy, gán 1 nếu nhỏ hơn	Lệnh thật
blt, bgt, ble, bge	So sánh lớn nhỏ	Giả lệnh, dùng $at
beqz, bnez	So với 0	Giả lệnh
j nhãn	Luôn nhảy	Lệnh thật

sltu so sánh không dấu. -1 không dấu là số rất lớn, nên kết quả khác slt.

38 Vòng lặp

Vòng lặp chỉ là một nhãn, một lệnh kiểm tra và một lệnh nhảy ngược lại.

```
main:   li   $s0, 1           # i = 1
        li   $s1, 100         # n = 100
        li   $s2, 0           # sum = 0
loop:   bgt  $s0, $s1, done   # i > n thì thoát
        add  $s2, $s2, $s0    # sum += i
        addi $s0, $s0, 1      # i++
        j    loop
done:   li   $v0, 1
        move $a0, $s2
        syscall
        li   $v0, 10
        syscall
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
5050
```

C	Hợp ngữ
while (cond)	Nhãn đầu, kiểm tra điều kiện ngược, thân, j về đầu
for (i...)	Khởi tạo, rồi giống while, tăng i cuối thân
do while	Thân trước, kiểm tra cuối, nhảy ngược khi đúng
break	j tới nhãn sau vòng lặp
continue	j tới phần tăng i

Đặt điều kiện ở cuối vòng lặp thì mỗi vòng bớt được một lệnh nhảy. Trình biên dịch luôn làm vậy, xem ở phần 5.

39 Duyệt mảng

Có hai cách: tính địa chỉ từ chỉ số, hoặc dùng con trỏ tăng dần. Cách con trỏ ít lệnh hơn trong mỗi vòng.

```
        .data
a:      .word 3, 9, 2, 7, 5
n:      .word 5
        .text
main:   la   $s0, a          # con trỏ tới phần tử đầu
        lw   $s1, n          # số phần tử
        li   $s2, 0          # tổng
        lw   $s3, 0($s0)     # lớn nhất
loop:   beqz $s1, done
        lw   $t0, 0($s0)
        add  $s2, $s2, $t0
        ble  $t0, $s3, next
        move $s3, $t0
next:   addi $s0, $s0, 4     # sang phần tử sau
        addi $s1, $s1, -1
        j    loop
done:   li   $v0, 10
        syscall
# Xem: $s2 $s3
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
$s2=26   $s3=9
```

Cách	Mỗi vòng cần	Số lệnh
Chỉ số a[i]	sll, add, lw, addi	Nhiều hơn
Con trỏ *p++	lw, addi	Ít hơn

Thói quen tốt: viết chú thích thanh ghi nào giữ biến nào ở đầu đoạn code. Hợp ngữ không có tên biến để nhắc bạn.

40 Xử lý chuỗi

Chuỗi là mảng byte kết thúc bằng số 0. Dùng lb, sb. Đổi chữ thường sang chữ hoa chỉ là trừ 32.

```
        .data
s:      .asciiz "code4life"
        .text
main:   la   $s0, s
        li   $s1, 0          # độ dài
loop:   lb   $t0, 0($s0)
        beqz $t0, done       # gặp số 0 thì dừng
        addi $s1, $s1, 1
        blt  $t0, 'a', skip  # không phải chữ thường
        bgt  $t0, 'z', skip
        addi $t0, $t0, -32   # sang chữ hoa
        sb   $t0, 0($s0)
skip:   addi $s0, $s0, 1     # chuỗi tăng 1, không phải 4
        j    loop
done:   li   $v0, 4
        la   $a0, s
        syscall
        li   $v0, 11
        li   $a0, ' '
        syscall
        li   $v0, 1
        move $a0, $s1
        syscall
        li   $v0, 10
        syscall
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
CODE4LIFE 9
```

- ✓ .asciiz tự thêm byte 0 ở cuối, .ascii thì không.
- ✓ Duyệt chuỗi tăng 1, duyệt mảng word tăng 4.
- ✓ lb mở rộng dấu, nên byte lớn hơn 127 sẽ thành số âm, hãy dùng lbu.

41 Gọi hàm: jal và jr

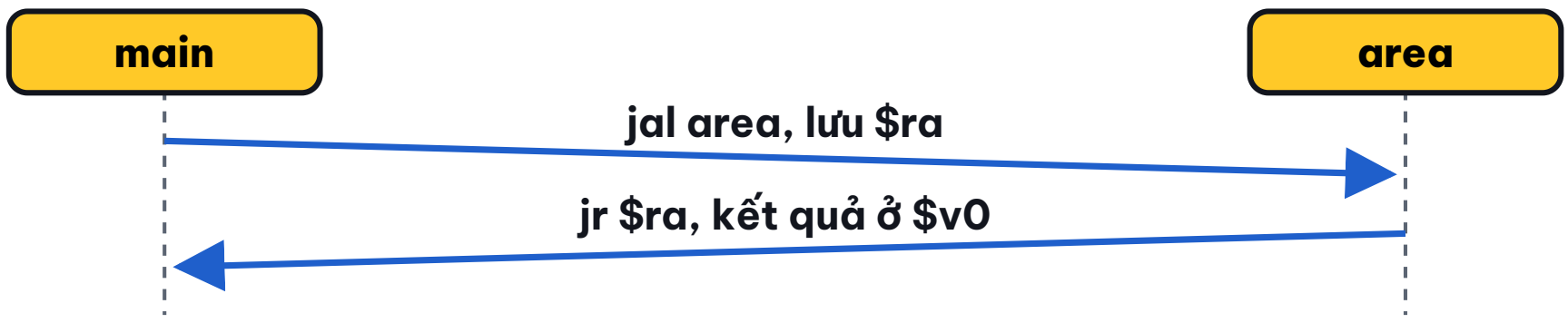
jal nhảy tới hàm và lưu địa chỉ quay về vào $ra. jr $ra quay lại. Tham số ở $a0 đến $a3, kết quả ở $v0.

```
main:   li   $a0, 6
        li   $a1, 7
        jal  area             # $ra = địa chỉ lệnh sau
        move $s0, $v0
        li   $a0, 10
        li   $a1, 3
        jal  area
        add  $s0, $s0, $v0
        li   $v0, 10
        syscall

area:   mul  $v0, $a0, $a1    # hàm area(w, h)
        jr   $ra
# Xem: $s0
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
$s0=72
```



Bước	Bên gọi	Hàm được gọi
Trước	Đặt tham số, lưu $t nếu cần	Lưu $ra, $s sẽ dùng
Sau	Đọc $v0	Đặt $v0, khôi phục, jr $ra

Đặt các hàm phía sau lệnh syscall 10, nếu không chương trình sẽ chạy tụt vào hàm.

42 Stack

Hàm gọi hàm khác thì jal ghi đè $ra. Phải cất $ra lên stack trước. Stack lớn dần về phía địa chỉ thấp.

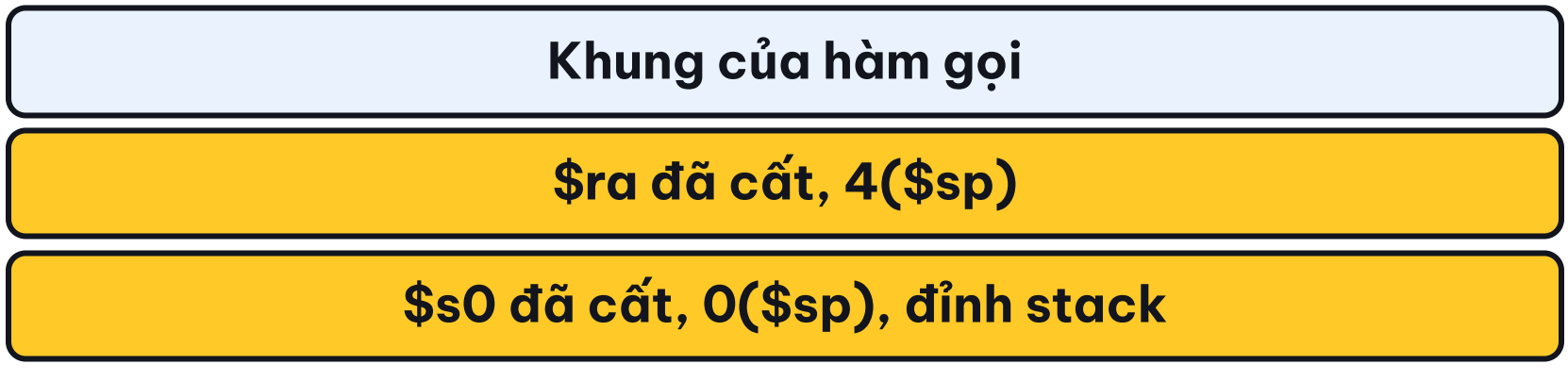
```
main:   li   $a0, 3
        jal  twice_sq
        move $s0, $v0
        li   $v0, 10
        syscall

twice_sq:                       # trả về 2 × a × a
        addi $sp, $sp, -8       # cấp 2 ô trên stack
        sw   $ra, 4($sp)        # cất $ra
        sw   $s0, 0($sp)        # cất $s0 để dùng
        move $s0, $a0
        jal  square
        add  $v0, $v0, $v0
        lw   $s0, 0($sp)        # khôi phục đúng thứ tự
        lw   $ra, 4($sp)
        addi $sp, $sp, 8
        jr   $ra

square: mul  $v0, $a0, $a0      # hàm lá không cần stack
        jr   $ra
# Xem: $s0
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
$s0=18
```



Thứ tự chuẩn: giảm $sp, lưu, làm việc, khôi phục, tăng $sp, jr $ra.

43 Đệ quy trong hợp ngữ

Đệ quy không có phép màu: mỗi lần gọi cất $ra và tham số lên stack. Đệ quy quá sâu thì tràn stack.

```
main:   li   $a0, 6
        jal  fact
        move $a0, $v0
        li   $v0, 1
        syscall
        li   $v0, 10
        syscall

fact:   addi $sp, $sp, -8
        sw   $ra, 4($sp)
        sw   $a0, 0($sp)
        bgt  $a0, 1, rec      # n > 1 thì đệ quy
        li   $v0, 1           # trường hợp cơ sở
        addi $sp, $sp, 8
        jr   $ra
rec:    addi $a0, $a0, -1
        jal  fact             # fact(n - 1)
        lw   $a0, 0($sp)      # lấy lại n
        lw   $ra, 4($sp)
        addi $sp, $sp, 8
        mul  $v0, $a0, $v0    # n × fact(n - 1)
        jr   $ra
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
720
```

Lỗi hay gặp	Hậu quả
Quên cất $ra	Lặp vô hạn, vì quay về chính nó
Quên lấy lại $a0	Tính sai, $a0 đã bị lần gọi sau ghi đè
Tăng $sp thiếu ở một nhánh	Stack lệch, jr nhảy lung tung

Mỗi tầng tốn 8 byte stack. Gọi fact(6) thì sâu nhất dùng 6 khung.

44 Dịch C sang MIPS

Bảng tra cho bài thi. Giả sử a, b, c ở $s0, $s1, $s2, mảng A có địa chỉ ở $s3, i ở $s4.

C	MIPS
`a = b + c`	`add $s0, $s1, $s2`
`a = b * 8`	`sll $s0, $s1, 3`
`a = A[2]`	`lw $s0, 8($s3)`
`A[3] = a`	`sw $s0, 12($s3)`

`a = A[i]`	`sll $t0, $s4, 2`
	`add $t0, $t0, $s3`
	`lw $s0, 0($t0)`

`if (a == b) X`	`bne $s0, $s1, skip`
	`X`
	`skip:`

`if (a < b) X`	`slt $t0, $s0, $s1`
	`beq $t0, $zero, skip`
	`X`
	`skip:`

`while (a ≠ 0) X`	`loop: beq $s0, $zero, end`
	`X`
	`j loop`
	`end:`

`f(a)`	`move $a0, $s0` `jal f`
`return a`	`move $v0, $s0` `jr $ra`

Điều kiện trong if thường bị đảo: nhảy qua thân khi điều kiện sai.

45 Ba định dạng lệnh

Mọi lệnh MIPS dài đúng 32 bit. Chỉ có ba kiểu, nên phần cứng giải mã rất đơn giản.



ĐỊA CHỈ, MÃ MÁY VÀ LỆNH

```
0x00400000  02324020  add  $t0, $s1, $s2
0x00400004  22510064  addi $s1, $s2, 100
0x00400008  8e680020  lw   $t0, 32($s3)
0x0040000c  ad490008  sw   $t1, 8($t2)
0x00400010  00094100  sll  $t0, $t1, 4
0x00400014  1109ffff  beq $t0, $t1, loop
0x00400018  08100005  j    loop
```

Kiểu	Dùng cho	Trường đặc biệt
R	add, sub, and, or, slt, sll, jr	op = 0, funct chọn phép toán
I	addi, lw, sw, beq, bne	Hằng 16 bit có dấu
J	j, jal	Địa chỉ 26 bit

Vị trí rs, rt giữ nguyên ở mọi kiểu, nên đọc thanh ghi và giải mã làm song song được. Thứ tự viết trong hợp ngữ khác thứ tự trong mã máy: add rd, rs, rt.

46 Mã hóa lệnh bằng tay

Dạng bài chắc chắn có. Tra bảng lấy op, funct và số thanh ghi, ghép bit rồi đổi sang hex.

```
REG = {"$t0": 8, "$t1": 9, "$s1": 17, "$s2": 18, "$s3": 19}
def r_type(rd, rs, rt, funct, shamt=0):
    w = 0 << 26 | REG[rs] << 21 | REG[rt] << 16
    return w | REG[rd] << 11 | shamt << 6 | funct

def i_type(op, rs, rt, imm):
    return op << 26 | REG[rs] << 21 | REG[rt] << 16 | imm & 0xFFFF
def show(w):
    b = f"{w:032b}"
    print(b[:6], b[6:11], b[11:16], b[16:], f"= {w:08X}")

show(r_type("$t0", "$s1", "$s2", 32))          # add $t0, $s1, $s2
show(i_type(35, "$s3", "$t0", 32))             # lw $t0, 32($s3)
show(i_type(8, "$s1", "$t0", -1))              # addi $t0, $s1, -1
```

KẾT QUẢ KHI CHẠY

```
000000 10001 10010 0100000000100000 = 02324020
100011 10011 01000 0000000000100000 = 8E680020
001000 10001 01000 1111111111111111 = 2228FFFF
```

Lệnh	op	funct	Lệnh	op
add	0	32	addi	8
sub	0	34	lw	35
and	0	36	sw	43
or	0	37	beq	4
slt	0	42	bne	5

Giải mã làm ngược lại: đổi hex ra bit, tách 6 bit đầu. Bằng 0 là kiểu R, xem funct. j là op 2, jal là op 3.

47 Mã hóa nhánh và lệnh nhảy

beq lưu khoảng cách tính bằng số lệnh, tính từ lệnh ngay sau nó. j lưu 26 bit giữa của địa chỉ đích.

ĐỊA CHỈ, MÃ MÁY VÀ LỆNH

```
0x00400000  21080001  addi $t0, $t0, 1
0x00400004  1509fffe  bne  $t0, $t1, main
0x00400008  11000002  beq  $t0, $zero, end
0x0040000c  00000000  nop
0x00400010  08100000  j    main
0x00400014  00000000  nop
```

```
def branch_offset(pc, target):
    return (target - (pc + 4)) // 4

def jump_field(target):
    return (target >> 2) & 0x3FFFFFF

print(branch_offset(0x00400004, 0x00400000))
print(branch_offset(0x00400008, 0x00400014))
print(hex(jump_field(0x00400000)))
```

KẾT QUẢ KHI CHẠY

```
-2
2
0x100000
```

Loại	Địa chỉ đích	Phạm vi
beq, bne	PC + 4 + độ dời × 4	Khoảng ±128 KB
j, jal	4 bit đầu của PC + 4, ghép 26 bit, ghép 00	Vùng 256 MB
jr	Toàn bộ 32 bit trong thanh ghi	Mọi nơi

Địa chỉ lệnh luôn chia hết cho 4, nên hai bit cuối bằng 0 và không cần lưu. Lệnh nhánh ngược có độ dời âm, dạng bù 2.

48 Giả lệnh

Một số lệnh chỉ có trong hợp ngữ. Trình hợp dịch đổi chúng thành lệnh thật, và thường dùng thanh ghi $at.

ĐỊA CHỈ, MÃ MÁY VÀ LỆNH

```
0x00400000  24080064  li   $t0, 100
0x00400004  3c011234  li   $t1, 0x12345678
0x00400008  34295678
0x0040000c  3c011001  la   $t2, x
0x00400010  342a0000
0x00400014  00085821  move $t3, $t0
0x00400018  0109082a  blt  $t0, $t1, main
0x0040001c  1420fff8
```

Giả lệnh	Thành lệnh thật
`move a, b`	`addu a, $zero, b`
`li a, 100`	`addiu a, $zero, 100`
`li a, số 32 bit`	`lui $at, cao rồi ori a, $at, thấp`
`la a, nhãn`	`lui $at, cao rồi ori a, $at, thấp`
`blt a, b, L`	`slt $at, a, b rồi bne $at, $zero, L`
`not a, b`	`nor a, b, $zero`

- ✓ Hằng số trong lệnh chỉ có 16 bit, nên số 32 bit phải nạp làm hai nửa.
- ✓ Đừng tự dùng $at, vì trình hợp dịch có thể ghi đè nó.
- ✓ Đề thi đếm số lệnh hoặc mã hóa thì phải dùng lệnh thật.

Dòng trắng ở cột lệnh là lệnh thật thứ hai của cùng một giả lệnh.

49 Chế độ địa chỉ

Cách một lệnh tìm toán hạng. MIPS có năm chế độ, rất ít so với x86.

Chế độ	Toán hạng ở đâu	Ví dụ
Thanh ghi	Trong thanh ghi	`add $t0, $t1, $t2`
Tức thì	Ngay trong lệnh	`addi $t0, $t1, 5`
Cơ sở	Thanh ghi + độ dời	`lw $t0, 8($s0)`
Tương đối PC	PC + 4 + độ dời × 4	`beq $t0, $t1, L`
Giả trực tiếp	Ghép với bit cao của PC	`j L`

Trên x86-64 có thêm	Ví dụ
Nền + chỉ số × tỉ lệ + độ dời	`mov eax, [rdi + rsi*4 + 8]`
Tương đối RIP	`lea rsi, [rel msg]`

Một lệnh x86 có thể tính cả a[i] trong một bước, còn MIPS cần ba lệnh. Đổi lại, phần cứng MIPS đơn giản hơn nhiều.

50 Lỗi hay gặp khi viết MIPS

Hợp ngữ không có trình biên dịch nhắc lỗi. Những lỗi dưới đây chiếm phần lớn thời gian debug của sinh viên.

Lỗi	Biểu hiện	Cách sửa
Quên nhân 4	Đọc sai phần tử, lỗi căn lề	sll $t0, $i, 2
Dùng lw cho chuỗi	Đọc 4 ký tự một lúc	Dùng lb, tăng 1
Nhầm giá trị và địa chỉ	Lỗi truy cập bộ nhớ	la lấy địa chỉ, lw lấy giá trị
Quên syscall 10	Chạy tụt vào hàm phía sau	Kết thúc main đúng cách
Không cất $ra	Lặp vô hạn	Cất lên stack
$t bị hàm con đổi	Biến tự thay đổi	Dùng $s, hoặc tự cất
Quên jr $ra	Chạy tụt sang hàm tiếp	Kết thúc hàm bằng jr
Đặt .word sau .asciiz	Lỗi căn lề	Thêm .align 2
Nhầm rs, rt	Điều kiện slt bị ngược	Đọc lại định nghĩa lệnh

CÁCH DEBUG TRONG MARS
Chạy từng bước, xem cột Registers và Data Segment thay đổi sau mỗi lệnh.

VIẾT BẰNG C TRƯỚC
Viết bằng C, chạy đúng rồi dịch từng dòng sang hợp ngữ. Nhanh hơn viết thẳng.

51 Thanh ghi x86-64

Đây là CPU trên chính máy tính của bạn. Có 16 thanh ghi 64 bit, mỗi thanh ghi truy cập được từng phần nhỏ.



64 bit	32 bit	16 bit	8 bit	Vai trò hay gặp
rax	eax	ax	al	Giá trị trả về, số hiệu syscall
rbx	ebx	bx	bl	Biến cần giữ
rcx	ecx	cx	cl	Bộ đếm, số bit dịch
rdx	edx	dx	dl	Nhân chia, tham số
rsi, rdi	esi, edi	si, di	sil, dil	Tham số, sao chép chuỗi
rsp, rbp	esp, ebp	sp, bp	spl, bpl	Đỉnh stack, khung stack
r8 đến r15	r8d	r8w	r8b	Thêm từ thời 64 bit

GHI 32 BIT
Ghi vào eax thì 32 bit cao của rax tự về 0. Ghi vào ax, al thì không.

RIP VÀ RFLAGS
rip là con trỏ lệnh, giống PC của MIPS. rflags chứa các cờ.

Tên ax, bx có từ thời 8086, sau mở rộng thành eax rồi rax. Vì vậy x86 trông không đều như MIPS.

52 Hello world bằng NASM

Không dùng thư viện nào, gọi thẳng kernel Linux. Số hiệu syscall đặt ở rax, tham số ở rdi, rsi, rdx.

```
section .data
msg:    db "Xin chào từ hợp ngữ", 10   ; 10 là xuống dòng
len:    equ $ - msg                    ; độ dài tính lúc dịch

section .text
global _start
_start:
    mov rax, 1          ; syscall write
    mov rdi, 1          ; stdout
    mov rsi, msg        ; địa chỉ chuỗi
    mov rdx, len        ; số byte
    syscall

    mov rax, 60         ; syscall exit
    mov rdi, 0          ; mã thoát
    syscall
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Xin chào từ hợp ngữ
```

```
nasm -f elf64 hello.asm -o hello.o
ld hello.o -o hello
./hello
```

Phần	Ý nghĩa
section .text	Lệnh
_start	Điểm vào khi không dùng thư viện C

Trên Windows không gọi syscall trực tiếp được như vậy, hãy chạy trong WSL.

53 Lệnh x86 cơ bản

Khoảng 20 lệnh này chiếm phần lớn mã máy bạn sẽ đọc. Lệnh có hai toán hạng, đích đứng trước theo cú pháp Intel.

Lệnh	Ý nghĩa	Ví dụ
mov	Chép dữ liệu	`mov rax, [rdi]`
lea	Tính địa chỉ, không đọc bộ nhớ	`lea rax, [rdi+rsi*4]`
add, sub	Cộng, trừ	`add rax, 8`
imul, idiv	Nhân, chia có dấu	`imul rax, rbx`
inc, dec, neg	Tăng, giảm, đổi dấu	`dec rcx`
and, or, xor, not	Logic	`xor eax, eax`
shl, shr, sar	Dịch bit	`shl rax, 3`
cmp, test	So sánh, chỉ đặt cờ	`cmp rax, rbx`
jmp, je, jne, jl	Nhảy	`jne .loop`
push, pop	Thao tác stack	`push rbx`
call, ret	Gọi hàm, trở về	`call printf`

Cú pháp	Ví dụ	Dùng ở
Intel	`mov rax, [rbx+8]`	NASM, tài liệu Intel, Windows
AT&T	`movq 8(%rbx), %rax`	Mặc định của gcc, objdump

xor eax, eax là cách đặt 0 ngắn nhất, gặp rất nhiều. Dùng gcc -masm=intel và objdump -M intel để xem theo cú pháp Intel.

54 Cờ và lệnh nhảy

cmp đặt cờ, lệnh nhảy đọc cờ. Phải chọn đúng lệnh cho số có dấu hay không dấu.

```
section .data
buf:    db "0", 10
section .text
global _start
_start:
    mov rbx, 5          ; đếm ngược
.next:
    mov rax, rbx
    add rax, '0'        ; thành ký tự
    mov [buf], al
    mov rax, 1
    mov rdi, 1
    mov rsi, buf
    mov rdx, 2
    syscall
    dec rbx             ; đặt cờ ZF
    jnz .next
    mov rax, 60
    xor rdi, rdi
    syscall
```

KẾT QUẢ CHẠY TRÊN LINUX

```
5
4
3
2
1
```

Sau cmp a, b	Có dấu	Không dấu
a < b	jl	jb
a > b	jg	ja

Bằng: je, khác: jne. Cờ chính: ZF bằng 0, SF âm, CF tràn không dấu, OF tràn có dấu.

55 Viết hàm hợp ngữ gọi từ C

Thực tế ít ai viết cả chương trình bằng hợp ngữ. Chỉ viết đoạn cần tối ưu, rồi gọi từ C.

```
global sum
section .text
sum:                    ; rdi = mảng, rsi = số phần tử
    xor rax, rax        ; tổng = 0
.loop:
    test rsi, rsi       ; hết phần tử chưa?
    jz .done
    add rax, [rdi]      ; cộng thẳng từ bộ nhớ
    add rdi, 8          ; long long là 8 byte
    dec rsi
    jmp .loop
.done:
    ret                 ; kết quả ở rax
```

KẾT QUẢ KHI GỌI TỪ C

```
14
```

```
#include <stdio.h>
long long sum(const long long *a, long long n);

int main(void) {
    long long a[] = {3, 1, 4, 1, 5};
    printf("%lld\n", sum(a, 5));
    return 0;
}
```

```
nasm -f elf64 sum.asm -o sum.o
gcc main.c sum.o -o demo
```

Tên hàm trong global phải trùng tên trong C. Code trên theo quy ước Linux, đã chạy thật qua một chương trình C.

56 Quy ước gọi hàm

Hợp đồng để code của mọi trình biên dịch gọi được nhau. Linux và Windows dùng quy ước khác nhau.

Nội dung	Linux, macOS (System V)	Windows x64
4 tham số đầu	rdi, rsi, rdx, rcx	rcx, rdx, r8, r9
Tham số 5, 6	r8, r9	Trên stack
Giá trị trả về	rax	rax
Hàm con phải giữ	rbx, rbp, r12 đến r15	Thêm rsi, rdi
Vùng đệm stack	Không có	32 byte cho bên gọi cấp
Số thực	xmm0 đến xmm7	xmm0 đến xmm3

```c
int g(int x);

int f(int a) {
    return g(a) + g(a + 1);
}
```

HỢP NGỮ DO GCC SINH RA

```
f:
    push    r14
    push    rbx
    sub     rsp, 8
    mov     ebx, edi
    call    g
    mov     r14d, eax
    lea     edi, [rbx+1]
    call    g
    add     r14d, eax
    mov     eax, r14d
    add     rsp, 8
    pop     rbx
    pop     r14
    ret
```

- Giá trị a phải sống qua lời gọi g, nên gcc cất vào rbx, là thanh ghi được giữ.
- Vì rbx phải trả lại, hàm push ở đầu và pop ở cuối.
- Stack phải căn lề 16 byte tại lệnh call, nên có sub rsp, 8.

Các đoạn hợp ngữ trong cẩm nang dịch theo quy ước Linux. Biên dịch trên Windows mà không có -mabi=sysv sẽ thấy rcx, rdx.

57 Khung stack

call đẩy địa chỉ trở về lên stack, ret lấy xuống. Biến cục bộ nằm ngay dưới đó. Đây là gốc của lỗi tràn bộ đệm.

```
int add(int a, int b) {
    int s = a + b;
    return s;
}
```

HỢP NGỮ DO GCC SINH RA

```
add:
    push    rbp
    mov     rbp, rsp
    mov     DWORD PTR [rbp-20], edi
    mov     DWORD PTR [rbp-24], esi
    mov     edx, DWORD PTR [rbp-20]
    mov     eax, DWORD PTR [rbp-24]
    add     eax, edx
    mov     DWORD PTR [rbp-4], eax
    mov     eax, DWORD PTR [rbp-4]
    pop     rbp
    ret
```

Khung của hàm gọi

Địa chỉ trở về, do call đẩy vào

rbp cũ, do push rbp

Biến cục bộ: [rbp-4], [rbp-20], [rbp-24]

Lệnh	Tương đương
push rbx	rsp = rsp - 8, rồi ghi rbx vào [rsp]
pop rbx	Đọc [rsp] vào rbx, rồi rsp = rsp + 8
call f	push địa chỉ lệnh kế tiếp, rồi jmp f
ret	pop vào rip

Ghi tràn một mảng cục bộ có thể đè lên địa chỉ trở về. Vì vậy trình biên dịch có stack canary, hệ điều hành có ASLR và vùng không thực thi.

58 Đọc hợp ngữ do gcc sinh ra

Cách học nhanh nhất: viết C rồi xem gcc dịch ra gì. Mức -O0 làm từng bước, -O1 trở lên thông minh hơn hẳn.

```c
int max(int a, int b) {
    return a > b ? a : b;
}
```

HỢP NGỮ DO GCC SINH RA

```
max:
    push    rbp
    mov     rbp, rsp
    mov     DWORD PTR [rbp-4], edi
    mov     DWORD PTR [rbp-8], esi
    mov     edx, DWORD PTR [rbp-8]
    mov     eax, DWORD PTR [rbp-4]
    cmp     edx, eax
    cmovge    eax, edx
    pop     rbp
    ret
```

```c
int max(int a, int b) {
    return a > b ? a : b;
}
```

HỢP NGỮ DO GCC SINH RA

```
max:
    cmp     esi, edi
    mov     eax, edi
    cmovge    eax, esi
    ret
```

```
gcc -O1 -S -masm=intel max.c -o max.s    # xem hợp ngữ
objdump -d -M intel a.out                # dịch ngược file đã biên dịch
```

Mức	Khi nào dùng
-O0	Debug, biến nằm trên stack, dễ theo dõi
-O2	Phát hành, tối ưu tốt
-O3	Tối ưu mạnh hơn, code có thể lớn hơn

cmovge là lệnh chép có điều kiện: không có nhánh nào để đoán sai. Xem Compiler Explorer (godbolt.org) để so nhiều trình biên dịch.

59 Vòng lặp trong mã máy

gcc đổi chỉ số a[i] thành con trỏ chạy, và đặt điều kiện ở cuối vòng. Đúng các mẹo ở phần MIPS.

```c
int sum(int *a, int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
```

HỢP NGỮ DO GCC SINH RA

```
sum:
    test    esi, esi
    jle     .L4
    mov     rax, rdi
    movsxd    rsi, esi
    lea     rcx, [rdi+rsi*4]
    mov     edx, 0
.L3:
    add     edx, DWORD PTR [rax]
    add     rax, 4
    cmp     rax, rcx
    jne     .L3
.L1:
    mov     eax, edx
    ret
.L4:
    mov     edx, 0
    jmp     .L1
```

Trong hợp ngữ	Ứng với C
`test esi, esi / jle`	n ≤ 0 thì bỏ qua vòng lặp
`lea rcx, [rdi+rsi*4]`	Địa chỉ ngay sau phần tử cuối
`add edx, [rax]`	s += *p
`add rax, 4`	p++
`cmp rax, rcx / jne`	Chưa tới cuối thì lặp lại

Thân vòng lặp chỉ còn 4 lệnh, và biến i đã biến mất. Viết tay hiếm khi hơn được trình biên dịch.

✓ Hãy tập trung vào thuật toán và cách truy cập bộ nhớ.

60 Trình biên dịch thông minh thế nào

Lệnh nhân, chia tốn nhiều chu kỳ. gcc thay bằng các lệnh rẻ hơn, kể cả phép chia cho hằng số.

```
int mul5(int x) {
    return x * 5;
}
int div2(int x) {
    return x / 2;
}
int div10(unsigned x) {
    return x / 10;
}
int mod8(unsigned x) {
    return x % 8;
}
```

HỢP NGỮ DO GCC SINH RA

```
mul5:
    lea     eax, [rdi+rdi*4]
    ret
div2:
    mov     eax, edi
    shr     eax, 31
    add     eax, edi
    sar     eax
    ret
div10:
    mov     eax, edi
    mov     edx, 3435973837
    imul     rax, rdx
    shr     rax, 35
    ret
mod8:
    mov     eax, edi
    and     eax, 7
    ret
```

Mẹo	Ý nghĩa
lea eax, [rdi+rdi*4]	Dùng mạch tính địa chỉ để tính x × 5
shr eax, 31 rồi add	Chỉnh để số âm làm tròn về 0 đúng như C
Nhân rồi dịch	3435973837 gần bằng 2^{35} / 10, nên x × số đó rồi dịch 35 bit là x / 10
and eax, 7	x % 8 chỉ là lấy 3 bit thấp

Vì x / 2 với số có dấu phải xử lý số âm, dùng unsigned đôi khi cho code gọn hơn khi giá trị không âm.

61 Con trỏ, mảng, struct

Kiểu dữ liệu không còn tồn tại trong mã máy. Mọi thứ chỉ là địa chỉ cộng độ dời.

```c
struct point { int x, y; };

int gety(struct point *p) {
    return p->y;
}
int get(int *a, long long i) {
    return a[i];
}
```

HỢP NGỮ DO GCC SINH RA

```
gety:
    mov     eax, DWORD PTR [rdi+4]
    ret
get:
    mov     eax, DWORD PTR [rdi+rsi*4]
    ret
```

Trong C	Trong hợp ngữ
p→y	Đọc tại p + độ dời của trường y
a[i]	Đọc tại a + i × cỡ phần tử
int	4 byte, lệnh dùng thanh ghi eax
unsigned, signed	Cùng bit, chỉ khác lệnh so sánh, chia

```c
#include <stdio.h>
struct a { char c; int i; char d; };
struct b { int i; char c; char d; };
int main(void) {
    printf("%zu %zu\n", sizeof(struct a), sizeof(struct b));
    return 0;
}
```

KẾT QUẢ BIÊN DỊCH VÀ CHẠY

```
12 8
```

Cùng ba trường, xếp khác thứ tự thì struct lớn khác nhau, vì trình biên dịch thêm byte đệm để căn lề. Xếp trường lớn lên trước sẽ tiết kiệm bộ nhớ.

62 switch và đệ quy trong mã máy

Hai điều bất ngờ: switch có thể không còn lệnh so sánh nào, còn đệ quy có thể biến thành vòng lặp.

```c
int f(int x) {
    switch (x) {
    case 0: return 11;
    case 1: return 22;
    case 2: return 37;
    case 3: return 45;
    case 4: return 58;
    case 5: return 63;
    default: return 0;
    }
}
```

HỢP NGỮ DO GCC SINH RA

```
f:
    xor     eax, eax
    cmp     edi, 5
    ja     .L1
    mov     edi, edi
    lea     rax, CSWTCH.1[rip]
    mov     eax, DWORD PTR [rax+rdi*4]
.L1:
    ret
CSWTCH.1:
```

```c
int fact(int n) {
    if (n <= 1)
        return 1;
    return n * fact(n - 1);
}
```

HỢP NGỮ DO GCC SINH RA

```
fact:
    mov     eax, 1
    cmp     edi, 1
    jle     .L1
.L2:
    imul     eax, edi
    sub     edi, 1
    cmp     edi, 1
    jne     .L2
.L1:
    ret
```

- ✓ switch nhiều case liên tiếp thành bảng tra hoặc bảng nhảy, một lần so sánh giới hạn rồi tra thẳng.
- ✓ Đệ quy ở trên không còn call: gcc đổi thành vòng lặp, nên không tốn stack.
- ✓ Ở mức -O0 hàm fact vẫn là đệ quy thật, giống phiên bản MIPS.

63 Dùng gdb

gdb cho phép dừng chương trình, xem thanh ghi, bộ nhớ và chạy từng dòng. Biên dịch với -g để có tên biến.

```
$ gcc -g -O0 g.c -o g.exe
$ gdb -q g.exe
(gdb) break sum
Breakpoint 1 at 0x140001770: file g.c, line 3.
(gdb) run
3           long long s = 0;
(gdb) info registers rcx rdx
rcx            0x5ffe40            6291008
rdx            0x5                 5
(gdb) x/5dg $rcx
0x5ffe40:       3       1
0x5ffe50:       4       1
0x5ffe60:       5
(gdb) next
(gdb) next
(gdb) next
(gdb) info locals
i = 0
s = 3
(gdb) bt
#0  sum (a=0x5ffe40, n=5) at g.c:4
#1  0x00007ff79ad717fb in main () at g.c:10
(gdb) finish
Value returned is $1 = 14
```

Lệnh	Việc
`break, run, continue`	Đặt điểm dừng, chạy, chạy tiếp
`next, step, stepi`	Chạy qua hàm, vào hàm, một lệnh máy
`info registers, x/5dg`	Xem thanh ghi, xem bộ nhớ

Kết quả thật trên Windows, nên tham số ở rcx, rdx. Linux là rdi, rsi. bt xem chuỗi gọi hàm.

64 Mã máy x86 trông thế nào?

Mỗi dòng hợp ngữ thành một dãy byte. Lệnh x86 dài ngắn khác nhau, trong khi lệnh MIPS luôn đúng 4 byte.

```
$ nasm -f win64 sum.asm -o sum.obj
$ objdump -d -M intel sum.obj

0: 48 31 c0         xor  rax,rax
3: 48 85 f6         test rsi,rsi
6: 74 0c            je   14
8: 48 03 07         add  rax,QWORD PTR [rdi]
b: 48 83 c7 08      add  rdi,0x8
f: 48 ff ce         dec  rsi
12: eb ef           jmp  3
14: c3              ret
```

Quan sát	Giải thích
Byte 48 đứng đầu	Tiền tố REX, báo toán hạng 64 bit
Lệnh dài 1 đến 4 byte	Phải giải mã xong mới biết lệnh sau bắt đầu ở đâu
74 0c	Nhảy tương đối: tiến 12 byte kể từ lệnh sau
eb ef	ef là -17 dạng bù 2, nhảy lùi
c3	ret chỉ tốn 1 byte

ĐỘ DÀI THAY ĐỔI
Code gọn hơn, cache lệnh chứa được nhiều hơn.

ĐỘ DÀI CỐ ĐỊNH
Giải mã nhiều lệnh song song dễ hơn. Chip ARM tận dụng điều này.

Bảng trên ghi lại từ objdump, chỉ lược bớt khoảng trắng. Nhìn lại trang trước để đối chiếu với mã nguồn.

65 Hợp ngữ 8086 cho môn học cũ

Một số trường và môn Vi xử lý vẫn dạy 8086 với emu8086. Đây là tổ tiên 16 bit của x86-64, tư duy giống hệt.

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG DB 'Xin chao$'          ; chuoi ket thuc bang dau $
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX                  ; nap thanh ghi doan
    MOV AH, 9                   ; ham 9: in chuoi
    LEA DX, MSG
    INT 21H
    MOV AH, 4CH                 ; ham 4Ch: ket thuc
    INT 21H
MAIN ENDP
END MAIN
```

Thành phần	Ý nghĩa
AX, BX, CX, DX	Thanh ghi 16 bit, chia được thành AH, AL
CS, DS, SS, ES	Thanh ghi đoạn
Địa chỉ vật lý	Đoạn × 16 + độ dời, tạo địa chỉ 20 bit, 1 MB
INT 21H	Gọi dịch vụ DOS, số hiệu đặt ở AH

AH = 1 đọc ký tự, 2 in ký tự, 9 in chuỗi, 4CH kết thúc. Đoạn code trên chưa được chạy thử, vì cần DOS hoặc emu8086.

66 MIPS, x86, ARM, RISC-V

Biết một tập lệnh là đọc được các tập lệnh khác. Cùng một phép tính a = b + c trên bốn kiến trúc:

Tập lệnh	Lệnh
MIPS	`add $t0, $t1, $t2`
RISC-V	`add a0, a1, a2`
ARM64	`add x0, x1, x2`
x86-64	`lea eax, [rdi+rsi]`

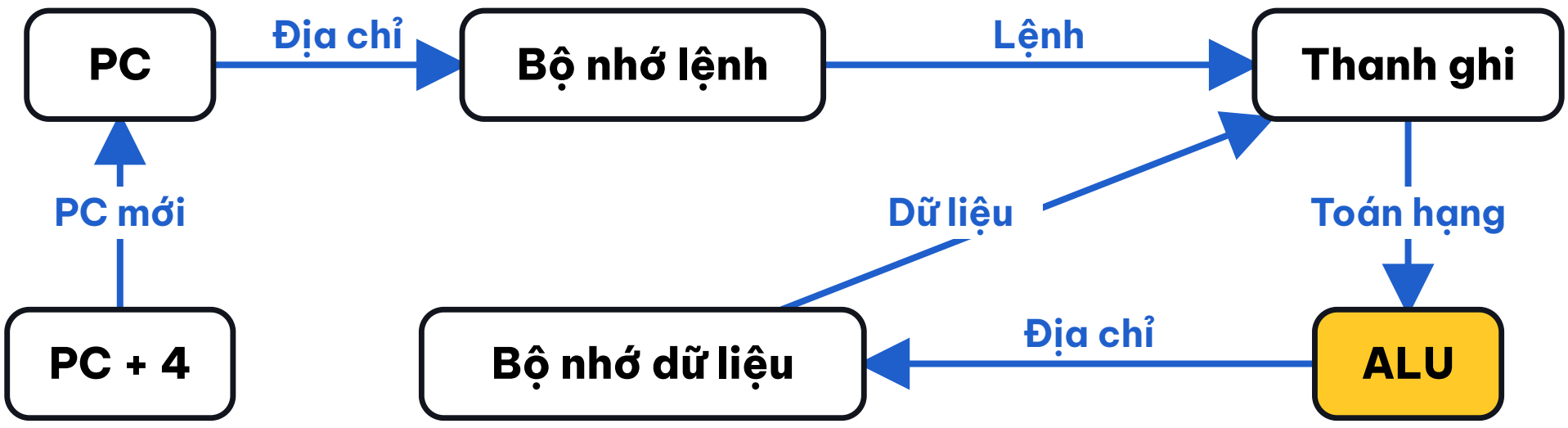
Tiêu chí	MIPS32	RISC-V	ARM64	x86-64
Kiểu	RISC	RISC	RISC	CISC
Thanh ghi	32	32	31	16
Độ dài lệnh	4 byte	4, có bản nén 2	4 byte	1 đến 15 byte
Thanh ghi 0	$zero	x0	xzr	Không có
Cờ trạng thái	Không	Không	Có	Có
Bản quyền	Đã mở	Mở	Cấp phép	Intel, AMD

Vì sao laptop chip ARM tiết kiệm pin? Một phần ở giải mã đơn giản, nhưng công nghệ chế tạo và thiết kế tổng thể mới là yếu tố lớn.

- ✓ Máy Apple đời mới, đa số điện thoại, nhiều server cloud chạy ARM.
- ✓ Docker image cũng phải khớp kiến trúc: linux/amd64 hay linux/arm64.
- ✓ RISC-V rất đáng học nếu bạn theo hướng chip, nhúng.

67 Đường dữ liệu đơn chu kỳ

Ghép các khối đã học thành CPU MIPS. Mỗi lệnh chạy xong trong đúng một chu kỳ xung nhịp.



Khối	Nhiệm vụ	Loại mạch
PC	Giữ địa chỉ lệnh hiện tại	Tuần tự
Bộ nhớ lệnh	Cho ra lệnh tại địa chỉ PC	Chỉ đọc
Tệp thanh ghi	Đọc 2, ghi 1 thanh ghi cùng lúc	Tuần tự
ALU	Tính toán, tính địa chỉ, so sánh	Tổ hợp
Bộ nhớ dữ liệu	Đọc cho lw, ghi cho sw	Tuần tự
Mở rộng dấu	Đổi hằng 16 bit thành 32 bit	Tổ hợp
Mux	Chọn nguồn dữ liệu ở các điểm rẽ	Tổ hợp

Hình đã lược bớt mux và khối mở rộng dấu. Khi học, hãy tự vẽ lại hình đầy đủ trong giáo trình ít nhất một lần.

68 Đường đi của từng lệnh

Lệnh khác nhau đi qua các khối khác nhau. Đây là chìa khóa để hiểu tín hiệu điều khiển và thời gian của lệnh.

Lệnh	Nạp	Đọc thanh ghi	ALU	Bộ nhớ	Ghi thanh ghi
add	Có	Có	Có		Có
lw	Có	Có	Có	Đọc	Có
sw	Có	Có	Có	Ghi	
beq	Có	Có	Có		
j	Có				

1 **add $t0, $s1, $s2**
Đọc hai thanh ghi, ALU cộng, ghi kết quả vào $t0.

2 **lw $t0, 8($s0)**
ALU tính $s0 + 8, đọc bộ nhớ, ghi vào $t0. Đây là lệnh dài nhất.

3 **sw $t0, 8($s0)**
ALU tính địa chỉ, ghi $t0 vào bộ nhớ.

4 **beq $t0, $t1, L**
ALU trừ hai số. Cờ Zero bằng 1 thì PC nhận địa chỉ nhánh.

Bài thi hay hỏi: lệnh X không dùng khối nào, tín hiệu nào không quan tâm. Bảng trên trả lời được đa số câu đó.

69 Tín hiệu điều khiển

Khối điều khiển đọc opcode rồi bật các tín hiệu. Đây là bảng kinh điển của CPU MIPS đơn chu kỳ.

Tín hiệu	R	lw	sw	beq
RegDst	1	0	X	X
ALUSrc	0	1	1	0
MemtoReg	0	1	X	X
RegWrite	1	1	0	0
MemRead	0	1	0	0
MemWrite	0	0	1	0
Branch	0	0	0	1
ALUOp	10	00	00	01

Tín hiệu	Bằng 1 nghĩa là
RegDst	Ghi vào thanh ghi rd, bằng 0 thì ghi vào rt
ALUSrc	Đầu vào thứ hai của ALU là hằng số
MemtoReg	Dữ liệu ghi về lấy từ bộ nhớ, không lấy từ ALU
Branch	Lệnh nhánh, kết hợp cờ Zero để chọn PC

X nghĩa là giá trị nào cũng được, vì kết quả không được ghi. ALUOp 10 thì khối điều khiển ALU xem thêm trường funct.

70 Chu kỳ xung nhịp đơn chu kỳ

Mọi lệnh dùng chung một chu kỳ, nên chu kỳ phải đủ cho lệnh chậm nhất. Lệnh nhanh cũng phải chờ cho hết.

```
delay = {"nạp": 200, "đọc reg": 100, "ALU": 200,
         "bộ nhớ": 200, "ghi reg": 100}
path = {"add": ["nạp", "đọc reg", "ALU", "ghi reg"],
        "lw":  ["nạp", "đọc reg", "ALU", "bộ nhớ", "ghi reg"],
        "sw":  ["nạp", "đọc reg", "ALU", "bộ nhớ"],
        "beq": ["nạp", "đọc reg", "ALU"],
        "j":   ["nạp"]}

time = {op: sum(delay[s] for s in p) for op, p in path.items()}
print(time)
T = max(time.values())
print("Chu kỳ:", T, "ps, tần số:", round(1e12 / T / 1e9, 2), "GHz")

mix = {"add": .45, "lw": .25, "sw": .10, "beq": .15, "j": .05}
avg = sum(mix[op] * time[op] for op in mix)
print("Mỗi lệnh chạy đúng thời gian của nó:", round(avg), "ps")
print("Lãng phí:", round(T / avg, 2), "lần")
```

KẾT QUẢ KHI CHẠY

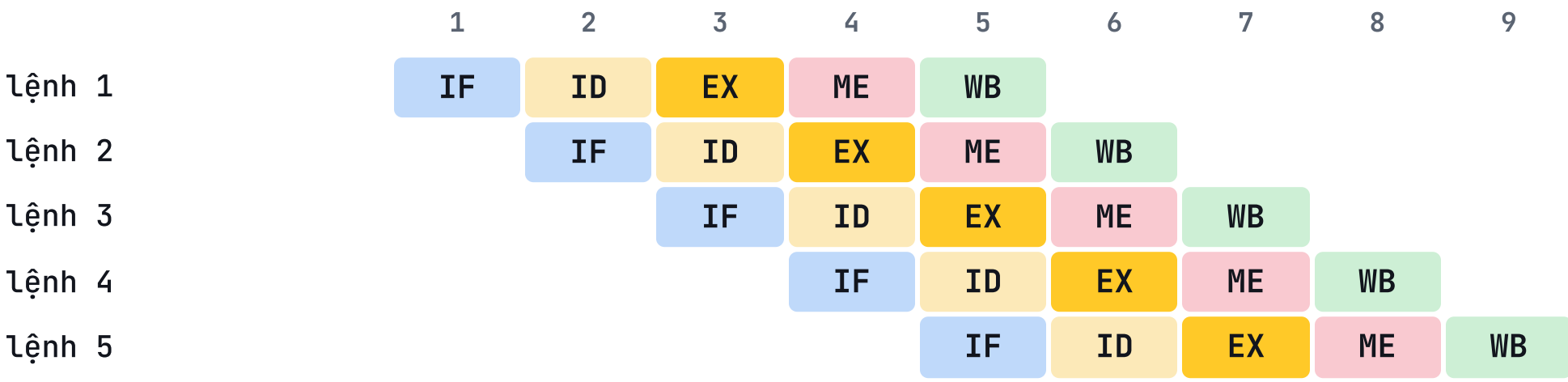
```
{'add': 600, 'lw': 800, 'sw': 700, 'beq': 500, 'j': 200}
Chu kỳ: 800 ps, tần số: 1.25 GHz
Mỗi lệnh chạy đúng thời gian của nó: 625 ps
Lãng phí: 1.28 lần
```

Không CPU thực tế nào dùng đơn chu kỳ. Cách giải quyết là pipeline: chu kỳ ngắn mà vẫn hoàn thành gần một lệnh mỗi chu kỳ.

Độ trễ và tỉ lệ lệnh trong code là số giả định theo kiểu đề bài hay gặp.

71 Pipeline là gì?

Giống giặt đồ: mẻ một vào máy sấy thì mẻ hai vào máy giặt. Mỗi lệnh không nhanh hơn, nhưng nhiều lệnh cùng chạy.




```python
def cycles(n, k):
    return k + n - 1                    # sau đó mỗi chu kỳ xong 1 lệnh

n, k = 1_000_000, 5
single = n * 800                        # đơn chu kỳ, mỗi lệnh 800 ps
piped = cycles(n, k) * 200              # pipeline, chu kỳ 200 ps
print("Chu kỳ:", cycles(n, k))
print(f"Tăng tốc: {single / piped:.2f} lần")
print(f"5 lệnh thôi: {5 * 800 / (cycles(5, k) * 200):.2f} lần")
```

KẾT QUẢ KHI CHẠY

```
Chu kỳ: 1000004
Tăng tốc: 4.00 lần
5 lệnh thôi: 2.22 lần
```

Khái niệm	Ý nghĩa
Độ trễ	Thời gian một lệnh đi hết đường ống. Không giảm
Thông lượng	Số lệnh xong mỗi đơn vị thời gian. Tăng lên
Chu kỳ	Bằng tầng chậm nhất, cộng thêm thời gian thanh ghi pipeline

Tăng tốc lý tưởng bằng số tầng khi các tầng đều nhau và không có xung đột. Tầng lệch nhau thì tăng tốc giảm.

72 Năm tầng pipeline MIPS

Mỗi tầng dùng một phần phần cứng. MIPS được thiết kế để pipeline dễ: lệnh cùng độ dài, ít định dạng.

Tầng	Việc
IF	Nạp lệnh, tăng PC
ID	Giải mã, đọc thanh ghi
EX	ALU tính toán hoặc tính địa chỉ
MEM	Đọc, ghi bộ nhớ dữ liệu
WB	Ghi kết quả về thanh ghi

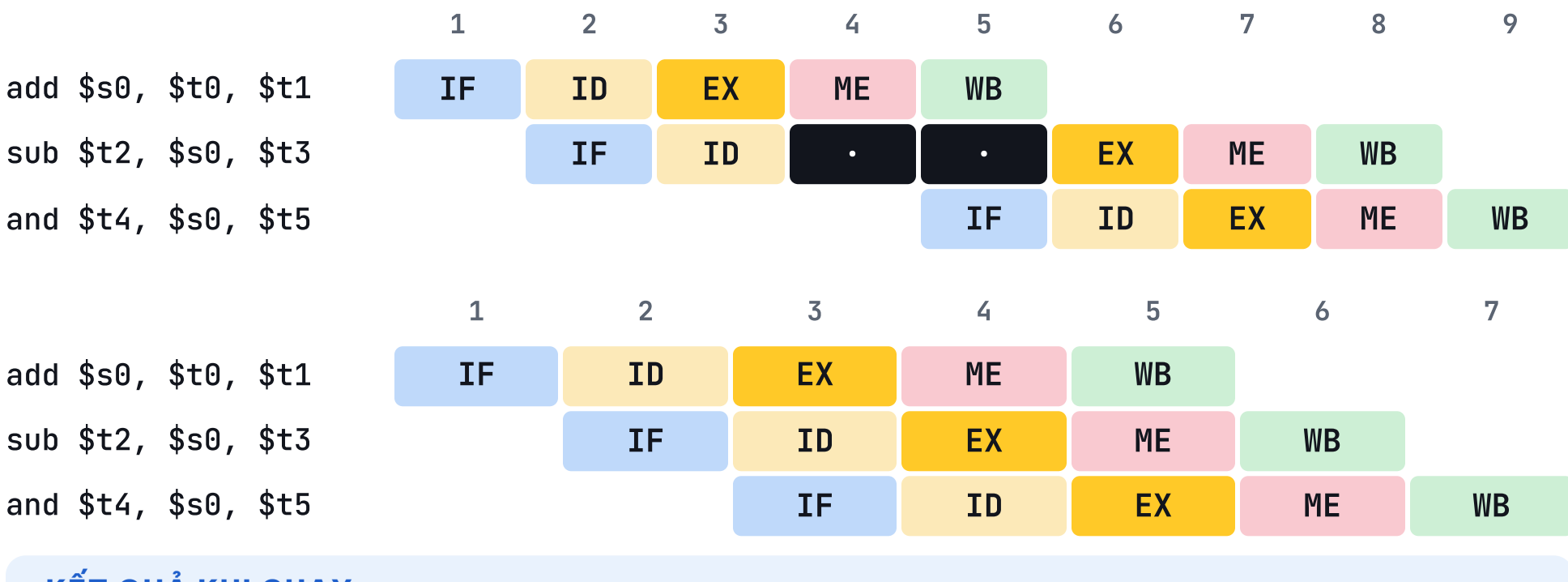


Loại xung đột	Nguyên nhân	Cách xử lý
Cấu trúc	Hai lệnh cần cùng phần cứng	Tách bộ nhớ lệnh, dữ liệu
Dữ liệu	Lệnh sau cần kết quả lệnh trước	Chuyển tiếp, chèn stall
Điều khiển	Chưa biết nhánh đi đâu	Dự đoán, xóa lệnh sai

Các thanh ghi pipeline IF/ID, ID/EX, EX/MEM, MEM/WB nằm giữa các tầng để giữ dữ liệu của từng lệnh.

73 Xung đột dữ liệu và chuyển tiếp

Lệnh sub cần $s0 ngay, trong khi add mãi tới WB mới ghi. Chuyển tiếp đưa kết quả từ sau ALU về thẳng đầu vào ALU.



KẾT QUẢ KHI CHẠY

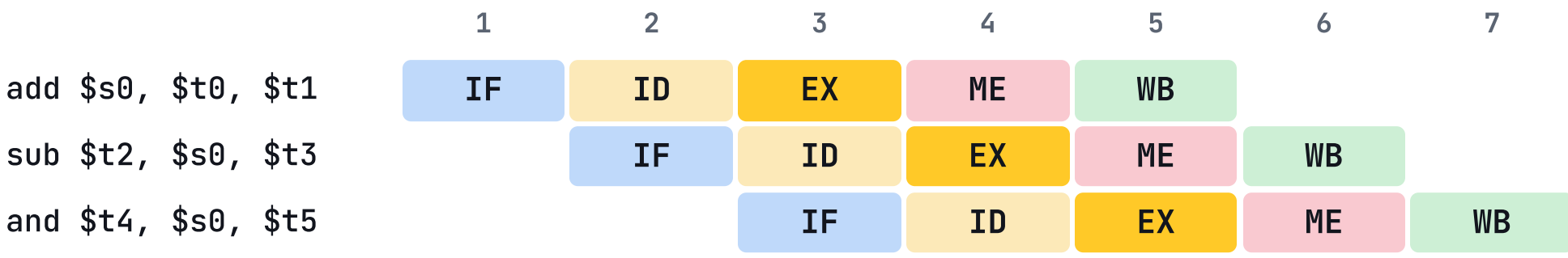
```
Không chuyển tiếp: 9 chu kỳ | có: 7 chu kỳ
```

Tình huống	Không chuyển tiếp	Có chuyển tiếp
Lệnh ALU rồi dùng ngay	2 stall	0 stall
lw rồi dùng ngay	2 stall	1 stall
Cách 1 lệnh	1 stall	0 stall
Cách 2 lệnh trở lên	0 stall	0 stall

Hình trên không chuyển tiếp, hình dưới có chuyển tiếp. Ô đen là stall. Bộ chuyển tiếp là các mux ở đầu vào ALU, chọn giá trị mới nhất.

74 Code mô phỏng pipeline

Mô hình gọn cho pipeline 5 tầng: với mỗi lệnh, tính chu kỳ sớm nhất được vào EX, phần chênh là stall.




```
def schedule(prog, forward):
    fetch, ready, total = 1, {}, 0
    for name, dst, srcs, load in prog:
        ex = fetch + 2                          # sớm nhất vào EX
        for s in srcs:
            ex = max(ex, ready.get(s, 0))
        stalls = ex - fetch - 2
        print(f"P: {name} | {fetch} IF ID {'-- ' * stalls}EX ME WB")
        if dst:                                 # EX sớm nhất
            ready[dst] = ex + 3
            if forward:
                ready[dst] = ex + 2 if load else ex + 1
        fetch += 1 + stalls
        total = ex + 2
    return total

prog = [("add $s0, $t0, $t1", "$s0", ["$t0", "$t1"], False),
        ("sub $t2, $s0, $t3", "$t2", ["$s0", "$t3"], False),
        ("and $t4, $s0, $t5", "$t4", ["$s0", "$t5"], False)]
print("Tổng:", schedule(prog, True), "chu kỳ")
```

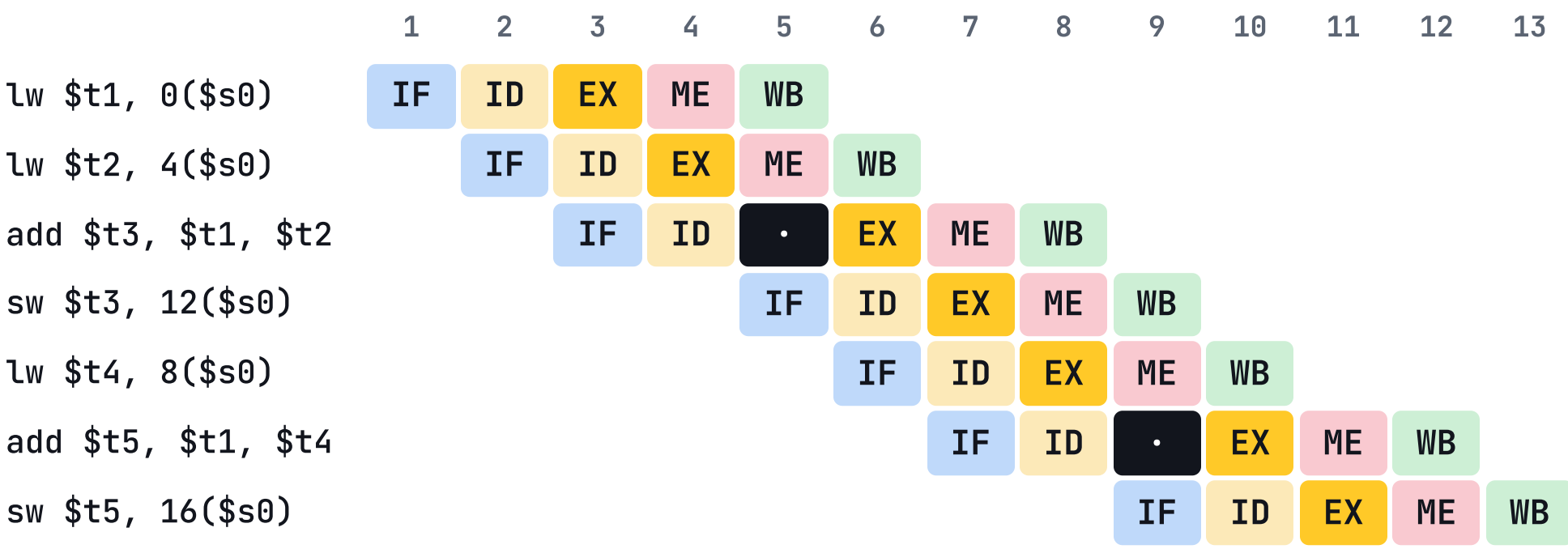
KẾT QUẢ KHI CHẠY

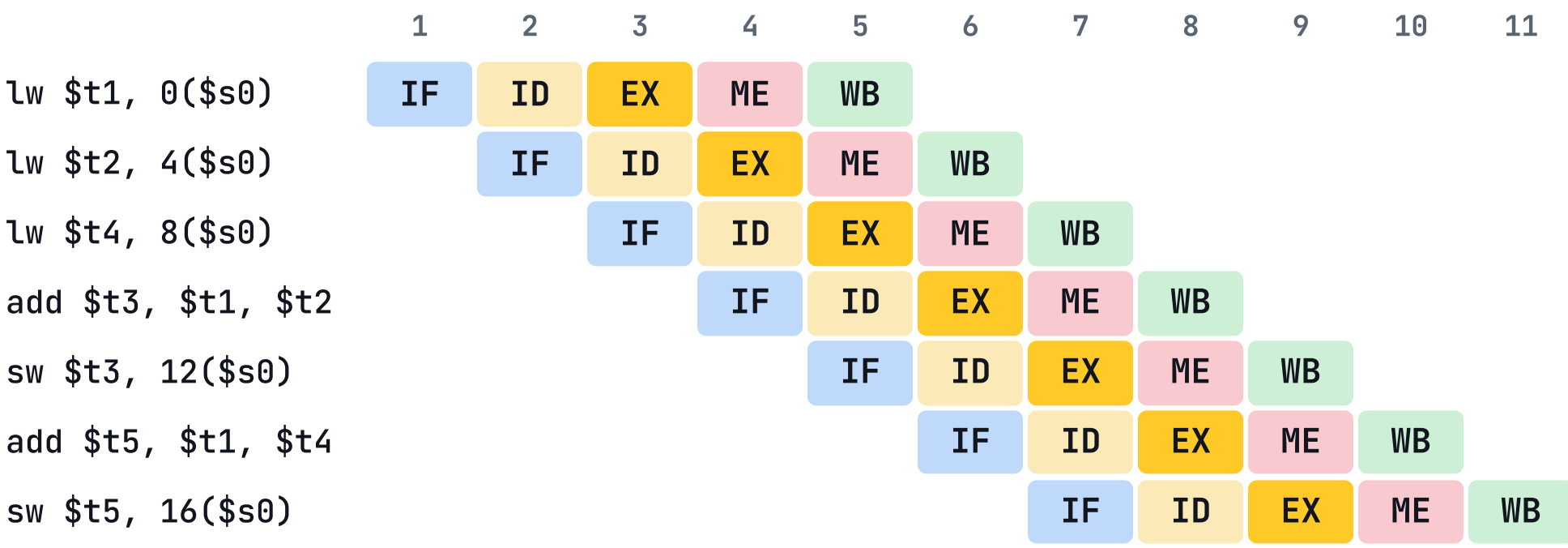
```
Tổng: 7 chu kỳ
```

ex + 1: kết quả ALU có ngay cho lệnh sau. ex + 2: kết quả lw có sau tầng MEM. ex + 3: không chuyển tiếp, chờ tới WB. Mô hình không xét nhánh.

75 Load-use và sắp xếp lệnh

Dữ liệu của lw có sau tầng MEM, nên dù chuyển tiếp vẫn trễ 1 chu kỳ. Trình biên dịch đổi thứ tự lệnh để lấp chỗ đó.





KẾT QUẢ KHI CHẠY

```
Thứ tự gốc: 13 chu kỳ, 2 stall
Đổi thứ tự: 11 chu kỳ, 0 stall
```

Chỉ đưa lw $t4 lên trước, hai stall biến mất. Không đổi kết quả, chỉ đổi thứ tự lệnh độc lập.

Công thức: số chu kỳ = 5 + (số lệnh - 1) + số stall.

76 Xung đột điều khiển

Khi nạp lệnh sau beq, CPU chưa biết có nhảy không. Đoán sai thì phải xóa các lệnh đã nạp nhầm.

```
def cpi(base, branch_rate, miss_rate, penalty):
    return base + branch_rate * miss_rate * penalty

for acc in (.50, .90, .95, .99):
    c = cpi(1, .20, 1 - acc, penalty=3)
    print(f"Đoán đúng {acc:.0%}: CPI = {c:.3f}")

print("Pipeline sâu, phạt 15 chu kỳ:",
      round(cpi(1, .20, .05, 15), 2))
```

KẾT QUẢ KHI CHẠY

```
Đoán đúng 50%: CPI = 1.300
Đoán đúng 90%: CPI = 1.060
Đoán đúng 95%: CPI = 1.030
Đoán đúng 99%: CPI = 1.006
Pipeline sâu, phạt 15 chu kỳ: 1.15
```

Cách xử lý	Ý tưởng
Dừng	Chờ biết kết quả nhánh. Đơn giản, chậm
Đưa so sánh lên ID	Thêm bộ so sánh để biết sớm, chỉ mất 1 chu kỳ
Khe trễ	Lệnh ngay sau nhánh luôn được chạy, MIPS thật có cơ chế này
Dự đoán tĩnh	Luôn đoán không nhảy, hoặc nhánh lùi thì nhảy
Dự đoán động	Nhớ lịch sử của từng nhánh

MARS mặc định tắt khe trễ. Trong đề thi, hãy xem đề có nói tới delayed branch không. Khoảng một phần năm số lệnh là nhánh, nên dự đoán rất quan trọng.

77 Dự đoán nhánh

Bộ đếm bão hòa 2 bit phải sai hai lần mới đổi ý, nên không bị lừa bởi lần thoát vòng lặp.




```python
def accuracy(outcomes, bits):
    top = 2 ** bits - 1
    state, right = top, 0
    for taken in outcomes:
        guess = state > top // 2
        right += guess == taken
        state = min(state + 1, top) if taken else max(state - 1, 0)
    return right / len(outcomes)

loop = ([True] * 9 + [False]) * 1000      # vòng lặp 10 lần
print(f"1 bit: {accuracy(loop, 1):.1%}")
print(f"2 bit: {accuracy(loop, 2):.1%}")

import random
rng = random.Random(1)
coin = [rng.random() < .5 for _ in range(10000)]
print(f"Nhánh ngẫu nhiên, 2 bit: {accuracy(coin, 2):.1%}")
```

KẾT QUẢ KHI CHẠY

```
1 bit: 80.0%
2 bit: 90.0%
Nhánh ngẫu nhiên, 2 bit: 50.4%
```

- ✓ 1 bit sai hai lần mỗi vòng lặp: lúc thoát, và lúc vào lại.
- ✓ CPU thật còn xét lịch sử nhiều nhánh gần đây.
- ✓ Nhánh thật sự ngẫu nhiên thì không bộ đoán nào giúp được.

Trạng thái trên hình là dự đoán nhảy (Có) hay không nhảy (Không). Dấu + nghĩa là chắc chắn.

78 Đo thật: mảng đã sắp xếp

Cùng một vòng lặp, cùng dữ liệu. Sắp xếp mảng trước làm nhánh dễ đoán, và tốc độ thay đổi hẳn.

```
for (int r = 0; r < 20000; r++)
    for (int i = 0; i < 32768; i++)
        if (a[i] >= 128)          // a[i] ngẫu nhiên từ 0 đến 255
            s += a[i];
```

Cách biên dịch	Chưa sắp xếp	Đã sắp xếp	Chênh lệch
-O1, tắt if-conversion	2,86 giây	0,44 giây	6,4 lần
-O1	1,06 giây	1,09 giây	Không đáng kể
-O2	0,42 giây	0,24 giây	1,7 lần
-O3	0,20 giây	0,22 giây	Không đáng kể

CÓ NHÁNH
Chưa sắp xếp thì nhánh đúng sai ngẫu nhiên, CPU đoán sai khoảng một nửa, mỗi lần xóa cả đường ống.

TRÌNH BIÊN DỊCH GỠ NHÁNH
Ở mức tối ưu cao, gcc dùng cmov hoặc SIMD, không còn nhánh để đoán sai.

- ✓ Bài học: đo trên chính cờ biên dịch bạn dùng, đừng tin một con số trên mạng.
- ✓ Viết code không nhánh chỉ có ích khi nhánh thật sự khó đoán.
- ✓ Xem bằng perf stat -e branch-misses trên Linux.

Kết quả đo thật trên Core i5-1035G1, gcc 16, Windows. Máy khác sẽ ra số khác.

79 CPU hiện đại làm gì thêm?

Pipeline 5 tầng chỉ là khởi đầu. CPU thật còn chạy nhiều lệnh một lúc, và không theo thứ tự trong code.

Kỹ thuật	Ý tưởng
Superscalar	Nạp và thực thi vài lệnh mỗi chu kỳ, CPI nhỏ hơn 1
Thực thi không theo thứ tự	Lệnh nào đủ dữ liệu thì chạy trước
Đổi tên thanh ghi	Xóa phụ thuộc giả do dùng lại tên thanh ghi
Thực thi suy đoán	Chạy trước theo nhánh dự đoán, sai thì hủy
Vi lệnh	x86 tách lệnh phức tạp thành lệnh nhỏ

```
// Phụ thuộc thành chuỗi
for (i...)
    s += a[i];
```

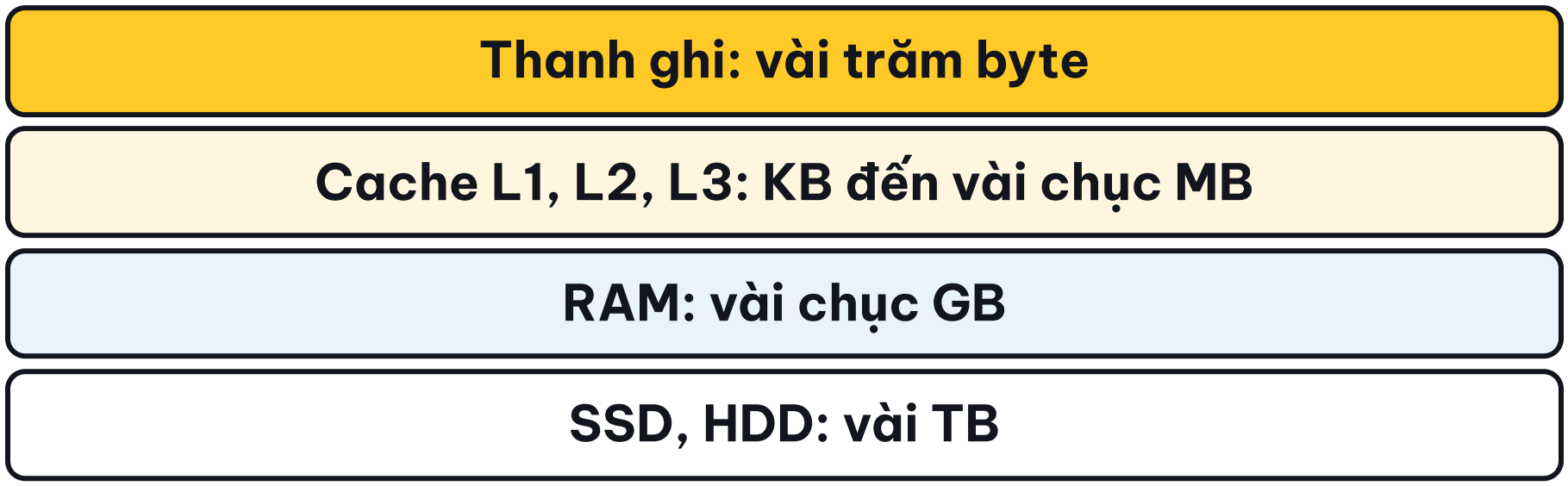
```
// Hai chuỗi độc lập
for (i...; i += 2) {
    s0 += a[i];
    s1 += a[i + 1];
}
```

Vì CPU tự tìm việc chạy song song, code ít phụ thuộc giữa các lệnh sẽ nhanh hơn dù số lệnh không đổi.

Đại lượng	Ý nghĩa
IPC	Số lệnh mỗi chu kỳ, bằng 1 / CPI
Pipeline sâu	Xung nhịp cao hơn, nhưng đoán sai bị phạt nặng hơn

80 Phân cấp bộ nhớ

Người dùng muốn bộ nhớ vừa nhanh, vừa lớn, vừa rẻ. Không có loại nào đủ cả ba, nên phải xếp thành nhiều tầng.



Tầng (máy thử nghiệm)	Dung lượng	Độ trễ cỡ
L1 dữ liệu	48 KB mỗi lõi	Vài chu kỳ
L2	512 KB mỗi lõi	Khoảng chục chu kỳ
L3	6 MB dùng chung	Vài chục chu kỳ
RAM	Vài GB trở lên	Hàng trăm chu kỳ

CỤC BỘ THỜI GIAN
Thứ vừa dùng dễ được dùng lại: biến đếm, lệnh trong vòng lặp.

CỤC BỘ KHÔNG GIAN
Thứ nằm gần dễ được dùng tiếp: phần tử kế tiếp của mảng.

Cache hiệu quả vì chương trình có tính cục bộ. Một lần trượt nạp cả khối 64 byte, nên các phần tử kề bên được dùng miễn phí.

Độ trễ chỉ thể hiện bậc độ lớn, tùy từng đời CPU.

81 SRAM, DRAM và băng thông

Cache dùng SRAM, RAM dùng DRAM. Hiểu băng thông giúp bạn biết vì sao nên cắm hai thanh RAM thay vì một.

Tiêu chí	SRAM	DRAM
Một bit cần	Khoảng 6 transistor	1 transistor, 1 tụ điện
Tốc độ	Rất nhanh	Chậm hơn
Làm tươi	Không cần	Phải làm tươi liên tục
Dùng cho	Cache	RAM

```python
def bandwidth(mt_per_s, channels):
    return mt_per_s * 8 * channels / 1000     # kênh rộng 8 byte

for name, mt in (("DDR4-3200", 3200), ("DDR5-5600", 5600)):
    print(f"{name}: 1 kênh {bandwidth(mt, 1):.1f} GB/s,"
          f" 2 kênh {bandwidth(mt, 2):.1f} GB/s")
```

KẾT QUẢ KHI CHẠY

```
DDR4-3200: 1 kênh 25.6 GB/s, 2 kênh 51.2 GB/s
DDR5-5600: 1 kênh 44.8 GB/s, 2 kênh 89.6 GB/s
```

Cắm 2 thanh 8 GB thường tốt hơn 1 thanh 16 GB, vì chạy được hai kênh. Lợi rõ nhất với máy dùng GPU tích hợp, vì GPU dùng chung RAM.

- ✓ Con số trên là băng thông đỉnh lý thuyết, thực tế thấp hơn.
- ✓ Độ trễ RAM gần như không giảm qua nhiều năm, chỉ băng thông tăng.
- ✓ Bộ nhớ ECC sửa được lỗi 1 bit, dùng trong server.

82 Cache ánh xạ trực tiếp

Mỗi địa chỉ chỉ vào đúng một dòng cache. Địa chỉ chia thành ba phần: tag, index và offset.




```python
def split(addr, lines=1024, block=16):
    offset_bits = block.bit_length() - 1
    index_bits = lines.bit_length() - 1
    offset = addr & (block - 1)
    index = addr >> offset_bits & (lines - 1)
    tag = addr >> (offset_bits + index_bits)
    return tag, index, offset

for a in (0x12345678, 0x00001234, 0x00005234):
    print(hex(a), "tag %d, index %d, offset %d" % split(a))
```

KẾT QUẢ KHI CHẠY

```
0x12345678 tag 18641, index 359, offset 8
0x1234 tag 0, index 291, offset 4
0x5234 tag 1, index 291, offset 4
```

Trường	Số bit	Công dụng
Offset	$\log_2$(cỡ khối)	Byte nào trong khối
Index	$\log_2$(số dòng)	Dòng nào trong cache
Tag	Số bit còn lại	So sánh để biết có đúng khối không

Dùng index chọn dòng, so tag và bit valid. Trúng thì dùng offset lấy dữ liệu. Trượt thì nạp cả khối từ tầng dưới.

Hai địa chỉ cuối có cùng index, khác tag, nên sẽ tranh nhau một dòng.

83 Giả lập cache

Dạng bài hay gặp: xác định trúng hay trượt. Đây là cache 4 dòng, mỗi khối 2 từ.

```python
def simulate(addrs, lines=4, block=2):
    tags, hits = [None] * lines, 0
    for a in addrs:
        blk = a // block                    # số thứ tự khối
        idx, tag = blk % lines, blk // lines
        hit = tags[idx] == tag
        hits += hit
        tags[idx] = tag
        print(f"{a:<3} khối {blk:<2} dòng {idx} tag {tag}",
              "trúng" if hit else "TRƯỢT")
    return hits / len(addrs)

rate = simulate([0, 1, 2, 8, 9, 0, 16, 0])
print(f"Tỉ lệ trúng: {rate:.0%}")
```

KẾT QUẢ KHI CHẠY

```
0   khối 0  dòng 0 tag 0 TRƯỢT
1   khối 0  dòng 0 tag 0 trúng
2   khối 1  dòng 1 tag 0 TRƯỢT
8   khối 4  dòng 0 tag 1 TRƯỢT
9   khối 4  dòng 0 tag 1 trúng
0   khối 0  dòng 0 tag 0 TRƯỢT
16  khối 8  dòng 0 tag 2 TRƯỢT
0   khối 0  dòng 0 tag 0 TRƯỢT
Tỉ lệ trúng: 25%
```

Loại trượt	Nguyên nhân	Giảm bằng
Bắt buộc	Lần đầu truy cập khối	Khối lớn hơn, nạp trước
Dung lượng	Cache quá nhỏ cho dữ liệu	Cache lớn hơn
Xung đột	Nhiều khối tranh một dòng	Tăng độ kết hợp

84 Cache kết hợp tập

Cho mỗi địa chỉ vài chỗ để chọn. Một tập có n dòng thì giảm hẳn xung đột, đổi lại phải so sánh tag song song.

```
def run(addrs, sets, ways):
    cache = [[] for _ in range(sets)]    # cuối là mới dùng
    misses = 0
    for a in addrs:
        s, line = a % sets, cache[a % sets]
        if a in line:
            line.remove(a)
        else:
            misses += 1
            if len(line) == ways:
                line.pop(0)              # thay khối cũ nhất
        line.append(a)
    return misses
addrs = [0, 8, 0, 8, 16, 0, 8, 16, 0, 8]
for name, sets, ways in (("Trực tiếp", 8, 1), ("2 đường", 4, 2),
                         ("4 đường", 2, 4), ("Toàn phần", 1, 8)):
    print(f"{name:10} trượt {run(addrs, sets, ways)}/{len(addrs)}")
```

KẾT QUẢ KHI CHẠY

```
Trực tiếp  trượt 10/10
2 đường    trượt 8/10
4 đường    trượt 3/10
Toàn phần  trượt 3/10
```

Tổ chức	Một khối nằm ở đâu	Đặc điểm
Trực tiếp	Đúng 1 dòng	Đơn giản, dễ xung đột
Kết hợp n đường	n dòng trong 1 tập	CPU thật thường dùng 8 đến 16 đường
Toàn phần	Bất kỳ dòng nào	Ít xung đột nhất, so sánh tốn kém

85 Tính kích thước cache

Bài thi hay cho dung lượng dữ liệu và hỏi tổng số bit. Mỗi dòng còn phải lưu thêm tag và bit valid.

```python
from math import log2

def cache_bits(data_bytes, block_bytes, ways=1, addr=32):
    lines = data_bytes // block_bytes
    sets = lines // ways
    offset = int(log2(block_bytes))
    index = int(log2(sets))
    tag = addr - index - offset
    total = lines * (8 * block_bytes + tag + 1)
    return offset, index, tag, total

o, i, t, total = cache_bits(16 * 1024, 16)
print(f"Offset {o}, index {i}, tag {t}")
print(f"Tổng {total / 1024:.0f} Kibit")
print(f"Gấp {total / (16 * 1024 * 8):.2f} lần phần dữ liệu")

o, i, t, total = cache_bits(32 * 1024, 64, ways=8)
print(f"L1 kiểu thật: offset {o}, index {i}, tag {t}")
```

KẾT QUẢ KHI CHẠY

```
Offset 4, index 10, tag 18
Tổng 147 Kibit
Gấp 1.15 lần phần dữ liệu
L1 kiểu thật: offset 6, index 6, tag 20
```

Tag = số bit địa chỉ - index - offset. Tổng bit = số dòng × (dữ liệu + tag + valid).

Cache ghi trễ cần thêm 1 bit dirty mỗi dòng. Tăng số đường gấp đôi thì index bớt 1 bit, tag thêm 1 bit.

86 Ghi vào cache

Đọc thì dễ. Khi ghi, phải quyết định khi nào cập nhật RAM, và có nạp khối vào cache không.

Chính sách khi trúng	Cách làm	Đặc điểm
Ghi xuyên	Ghi cả cache và RAM	Đơn giản, tốn băng thông
Ghi trễ	Chỉ ghi cache, đánh dấu dirty	Nhanh, phức tạp hơn

Chính sách khi trượt	Cách làm
Có cấp phát	Nạp khối vào cache rồi ghi
Không cấp phát	Ghi thẳng xuống RAM

CẶP HAY DÙNG
Ghi trễ đi với có cấp phát. Ghi xuyên đi với không cấp phát.

BỘ ĐỆM GHI
CPU đưa dữ liệu vào bộ đệm rồi chạy tiếp, không chờ RAM.

```
writes, hits = 1000, 950
through = writes                    # mỗi lần ghi đều xuống RAM
back = writes - hits                # chỉ khối bị đuổi mới ghi
print("Ghi xuyên:", through, "lần ghi RAM")
print("Ghi trễ:", back, "lần ghi RAM")
```

KẾT QUẢ KHI CHẠY

```
Ghi xuyên: 1000 lần ghi RAM
Ghi trễ: 50 lần ghi RAM
```

87 AMAT và CPI có cache

Thời gian truy cập trung bình cho biết cache hiệu quả đến đâu. Tỉ lệ trượt tăng nhẹ cũng có thể làm CPI tăng rất nhiều.

AMAT = Thời gian trúng + Tỉ lệ trượt × Thời gian phạt

```
def amat(hit, miss_rate, penalty):
    return hit + miss_rate * penalty

print("Một tầng:", amat(1, .05, 100), "chu kỳ")
l2 = amat(12, .20, 100)                    # L2 làm tầng phạt của L1
print("Thêm L2:", amat(1, .05, l2), "chu kỳ")

def cpi(base, i_miss, d_miss, mem_ratio, penalty):
    return base + i_miss * penalty + mem_ratio * d_miss * penalty

c = cpi(1, .02, .04, .36, 100)
print("CPI thật:", round(c, 2))
print(f"CPU mất {(c - 1) / c:.0%} thời gian chờ bộ nhớ")
print("Nhanh gấp đôi mà bộ nhớ không đổi:",
      round(c / cpi(1, .02, .04, .36, 200) * 2, 2), "lần")
```

KẾT QUẢ KHI CHẠY

```
Một tầng: 6.0 chu kỳ
Thêm L2: 2.6 chu kỳ
CPI thật: 4.44
CPU mất 77% thời gian chờ bộ nhớ
Nhanh gấp đôi mà bộ nhớ không đổi: 1.13 lần
```

Đại lượng	Công thức
AMAT nhiều tầng	Hit L1 + Miss L1 × (Hit L2 + Miss L2 × Phạt RAM)
Chu kỳ dừng do dữ liệu	Tỉ lệ lệnh nạp, lưu × tỉ lệ trượt × phạt

CPU nhanh gấp đôi thì phạt tính theo chu kỳ cũng gấp đôi, nên tốc độ thật tăng ít hơn 2 lần. Đây là bức tường bộ nhớ.

88

Đo thật: duyệt hàng, duyệt cột

Đây là lời giải câu đố ở bìa. Mảng C lưu theo hàng. Duyệt theo cột thì mỗi lần nhảy 16 KB, dùng 4 byte rồi bỏ cả khối.

```c
#define N 4096
static int a[N][N];                 // 64 MB, lớn hơn mọi cache

for (i = 0; i < N; i++)             // duyệt theo hàng
    for (j = 0; j < N; j++)
        s += a[i][j];

for (j = 0; j < N; j++)             // duyệt theo cột
    for (i = 0; i < N; i++)
        s += a[i][j];
```

```
$ gcc -O1 matrix.c -o matrix && ./matrix
hang 0.010 s, cot 0.247 s, gap 24.7 lan
hang 0.010 s, cot 0.243 s, gap 24.3 lan
hang 0.012 s, cot 0.252 s, gap 21.0 lan
```

THEO HÀNG

Khối 64 byte chứa 16 số int, nên tỉ lệ trượt chỉ 6,25%. CPU còn tự nạp trước khối tiếp theo.

THEO CỘT

Mỗi bước cách nhau 16 KB, nên tỉ lệ trượt gần 100%. Khối bị đuổi trước khi kịp dùng lại.

Kết quả đo thật trên Core i5-1035G1. Python, Java cũng chịu tác động này với mảng số liên tục như NumPy. Fortran, MATLAB lưu theo cột nên chiều ngược lại.

89 Viết code thân thiện với cache

Cùng độ phức tạp $O(n^3)$, chỉ đổi thứ tự vòng lặp, phép nhân ma trận nhanh gấp 8 lần.

```
// ijk: b[k][j] đi theo cột
for (i...) for (j...) {
    double s = 0;
    for (k...)
        s += a[i][k] * b[k][j];
    c[i][j] = s;
}
```

```
// ikj: mọi mảng đi theo hàng
for (i...) for (k...) {
    double v = a[i][k];
    for (j...)
        c[i][j] += v * b[k][j];
}
```

```
$ gcc -O2 mm.c -o mm && ./mm        # N = 1024
ijk 6.99 s, ikj 0.87 s, gap 8.0 lan
ijk 6.88 s, ikj 0.73 s, gap 9.4 lan
```

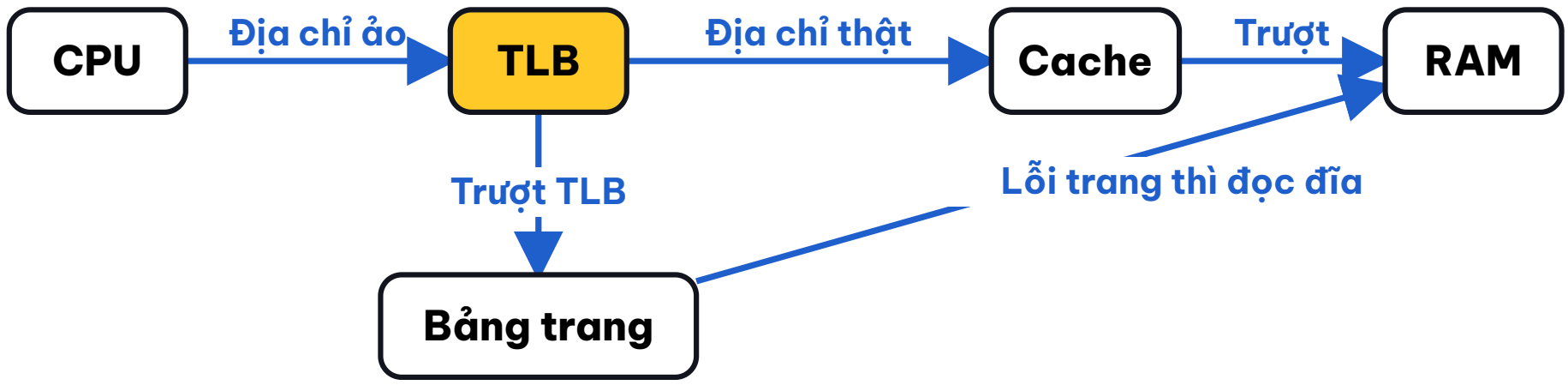
Kỹ thuật	Ý nghĩa
Duyệt tuần tự	Đi theo thứ tự lưu trong bộ nhớ
Chia khối (tiling)	Xử lý từng khối nhỏ vừa cache
Mảng thay vì danh sách liên kết	Phần tử nằm liền nhau, CPU nạp trước được
Tách trường hay dùng	Một mảng riêng cho trường cần duyệt
Cấu trúc nhỏ gọn	Nhiều phần tử hơn trong một khối

Vì vậy std::vector thường nhanh hơn std::list, kể cả khi chèn giữa danh sách vừa phải. Đừng đoán, hãy đo.

Kết quả đo thật trên máy thử nghiệm. Thư viện như OpenBLAS còn nhanh hơn nhiều nhờ chia khối và SIMD.

90 Bộ nhớ ảo và TLB

Bộ nhớ ảo là một tầng nữa trong phân cấp: RAM làm cache cho đĩa. Phần cứng dịch địa chỉ ảo ở mọi lần truy cập.



So sánh	Cache	Bộ nhớ ảo
Đơn vị	Khối, thường 64 byte	Trang, thường 4 KB
Khi trượt	Trượt cache, phạt hàng trăm chu kỳ	Lỗi trang, phạt hàng triệu chu kỳ
Xử lý bởi	Phần cứng	Hệ điều hành
Vị trí khối	Hạn chế theo số đường	Bất kỳ khung nào

```
page, tlb_entries = 4096, 64
print("TLB phủ được:", page * tlb_entries // 1024, "KB")
print("Với huge page 2 MB:", 2 * tlb_entries, "MB")
```

KẾT QUẢ KHI CHẠY

```
TLB phủ được: 256 KB
Với huge page 2 MB: 128 MB
```

Duyệt dữ liệu rải rác trên nhiều trang vừa trượt cache, vừa trượt TLB. Đó là một phần lý do duyệt cột rất chậm.

Chi tiết phân trang, thay trang, thrashing có trong Cẩm nang Hệ điều hành của Code4Life. Số mục TLB ở trên chỉ là ví dụ.

91 Bus và các chuẩn kết nối

Dữ liệu đi giữa các khối qua bus. Biết tốc độ từng chuẩn giúp chọn đúng linh kiện và tìm đúng nút cổ chai.

Chuẩn	Tốc độ lý thuyết	Dùng cho
SATA III	6 Gb/s	SSD 2,5 inch, HDD
PCIe 3.0 x4	Khoảng 4 GB/s	SSD NVMe đời cũ
PCIe 4.0 x4	Khoảng 8 GB/s	SSD NVMe phổ biến
PCIe 5.0 x4	Khoảng 16 GB/s	SSD đời mới
USB 3.2 Gen 2	10 Gb/s	Ổ cứng ngoài
USB4, Thunderbolt 4	40 Gb/s	Dock, ổ tốc độ cao

```python
def seconds(gb, gbit_per_s, efficiency=.8):
    return gb * 8 / (gbit_per_s * efficiency)

ports = (("USB 2.0", .48), ("USB 3.2 Gen 1", 5), ("USB 3.2 Gen 2", 10))
for name, speed in ports:
    print(f"{name:14} chép 20 GB: {seconds(20, speed):6.0f} giây")
```

KẾT QUẢ KHI CHẠY

```
USB 2.0        chép 20 GB:    417 giây
USB 3.2 Gen 1  chép 20 GB:     40 giây
USB 3.2 Gen 2  chép 20 GB:     20 giây
```

- ✓ Gb là gigabit, GB là gigabyte. Chia 8 để đổi.
- ✓ Cắm ổ nhanh vào cổng chậm thì tốc độ theo cổng chậm.
- ✓ Cáp USB-C không đảm bảo tốc độ: cùng hình dạng nhưng khác chuẩn bên trong.

Hiệu suất 80% trong code là giả định. Tốc độ thực tế còn phụ thuộc ổ đĩa và kích thước file.

92 Polling, ngắt và DMA

Ba cách CPU làm việc với thiết bị. Bài thi hay yêu cầu tính phần trăm thời gian CPU bị chiếm.

```python
def polling_share(polls_per_s, cycles_per_poll, hz):
    return polls_per_s * cycles_per_poll / hz

hz = 1e9
print(f"Chuột: {polling_share(30, 400, hz):.4%}")

rate, chunk = 40e6, 16                  # đĩa 40 MB/s, mỗi lần 16 byte
print(f"Đĩa polling: {polling_share(rate / chunk, 400, hz):.0%}")

block = 8192                            # DMA: 8 KB ngắt 1 lần
setup = 2000
print(f"Đĩa DMA: {polling_share(rate / block, setup, hz):.2%}")
```

KẾT QUẢ KHI CHẠY

```
Chuột: 0.0012%
Đĩa polling: 100%
Đĩa DMA: 0.98%
```

Cách	Hoạt động	Hợp với
Polling	CPU liên tục hỏi trạng thái	Thiết bị rất nhanh hoặc đơn giản
Ngắt	Thiết bị báo khi có việc	Bàn phím, sự kiện rời rạc
DMA	Bộ điều khiển tự chép vào RAM, xong mới ngắt	Đĩa, mạng, truyền khối lớn

Với DMA, CPU chỉ đặt lệnh ở đầu và nhận một ngắt ở cuối. Trong lúc truyền, CPU làm việc khác.

93 Đa lõi và phân loại Flynn

Khi không tăng được xung nhịp, chip có nhiều lõi hơn. Nhưng chỉ phần mềm viết song song mới hưởng lợi.

Loại	Ý nghĩa	Ví dụ
SISD	Một lệnh, một dữ liệu	CPU đơn lõi đời cũ
SIMD	Một lệnh, nhiều dữ liệu	AVX, GPU
MISD	Nhiều lệnh, một dữ liệu	Hầu như không có
MIMD	Nhiều lệnh, nhiều dữ liệu	CPU đa lõi, cụm máy

Khái niệm	Ý nghĩa
Lõi vật lý	Có đủ pipeline, ALU, cache L1 riêng
Siêu phân luồng (SMT)	Một lõi chạy hai luồng, dùng chung tài nguyên
Lõi P, lõi E	Lõi hiệu năng và lõi tiết kiệm điện
Bộ nhớ chung	Các lõi dùng chung RAM, cần cache nhất quán

Task Manager báo 8 CPU trên máy 4 lõi, 8 luồng. Hai luồng trên một lõi không nhanh gấp đôi, vì tranh nhau ALU và cache.

Xem lại định luật Amdahl: phần tuần tự luôn chặn trần tăng tốc của đa lõi.

94 Cache nhất quán và false sharing

Mỗi lõi có cache riêng, nên phải đồng bộ. Hai luồng ghi hai biến khác nhau trong cùng một khối vẫn tranh nhau cả khối.

Trạng thái MESI	Ý nghĩa
Modified	Chỉ cache này có, đã sửa, khác RAM
Exclusive	Chỉ cache này có, giống RAM
Shared	Nhiều cache cùng có, chỉ đọc
Invalid	Không dùng được

```
struct { long long a, b; } near;            // cùng một khối 64 byte
struct { long long a; char pad[248]; long long b; } far;

void *work(void *p) {                       // 2 luồng, 2 lõi
    for (long i = 0; i < 100000000; i++)
        __atomic_fetch_add((long long *)p, 1, __ATOMIC_RELAXED);
    return NULL;
}
```

```
cung dong cache 2.71 s, khac dong 0.69 s, gap 3.9 lan
cung dong cache 2.78 s, khac dong 0.70 s, gap 4.0 lan
cung dong cache 2.74 s, khac dong 0.66 s, gap 4.1 lan
```

Hai luồng không dùng chung biến nào, mà vẫn chậm gấp 4 lần. Khối cache bị chuyền qua lại giữa hai lõi sau mỗi lần ghi.

- ✓ Cách sửa: thêm byte đệm hoặc căn lề biến theo 64 byte.
- ✓ Gặp ở bộ đếm theo từng luồng, mảng kết quả chia cho các luồng.

Đo thật trên máy thử nghiệm, hai luồng gắn vào lõi 0 và lõi 2. Với biến volatile không nguyên tử, chênh lệch đo được nhỏ hơn nhiều, khoảng 1,2 lần.

95 SIMD: một lệnh, nhiều dữ liệu

Thanh ghi 256 bit chứa 8 số float. Một lệnh cộng cả 8 số. Đây là cách tăng tốc rẻ nhất cho xử lý ảnh, âm thanh, AI.

```
void add(float *restrict x, const float *restrict y,
         const float *restrict z) {
    for (int i = 0; i < 4096; i++)
        x[i] = y[i] * 2.0f + z[i];
}
```

Biên dịch	Lệnh dùng	Thời gian	Nhanh hơn
-O2, tắt vector	Từng số	2,14 giây	1 lần
-O3 -msse2	4 số một lúc	0,55 giây	3,9 lần
-O3 -mavx2 -mfma	8 số một lúc	0,28 giây	7,6 lần

```
float s0 = y[0] * 2 + z[0];
float s1 = y[1] * 2 + z[1];
// ... 8 lần
```

```
vmovups ymm0, [y]
vfmadd213ps ymm0, ymm1, [z]
vmovups [x], ymm0
```

- ✓ restrict báo các mảng không chồng lên nhau, giúp gcc dám vector hóa.
- ✓ Code có nhánh, truy cập rải rác khó vector hóa.
- ✓ NumPy, PyTorch nhanh một phần nhờ SIMD bên dưới.
- ✓ -march=native tận dụng hết CPU, nhưng file chạy có thể không chạy trên máy cũ.

Đo thật, lặp 1 triệu lần trên mảng 4096 số. Khung hợp ngữ bên phải chỉ minh họa ý tưởng, không phải toàn bộ code gcc sinh ra.

96 GPU khác CPU thế nào?

CPU tối ưu để một luồng chạy thật nhanh. GPU tối ưu để hàng nghìn luồng đơn giản chạy cùng lúc.

Tiêu chí	CPU	GPU
Số lõi	Vài đến vài chục lõi mạnh	Hàng nghìn đơn vị nhỏ
Tối ưu cho	Độ trễ	Thông lượng
Cache, dự đoán nhánh	Rất lớn, rất tốt	Ít hơn nhiều
Che độ trễ bộ nhớ	Cache	Chuyển sang luồng khác
Hợp với	Logic phức tạp, nhiều nhánh	Ma trận, ảnh, AI

- AI là nhân ma trận khổng lồ, rất hợp GPU, nên GPU trở thành trung tâm của ngành AI.
- Chép dữ liệu qua PCIe có thể tốn hơn cả phần tính, đừng chuyển việc nhỏ sang GPU.
- Tensor core là khối chuyên nhân ma trận độ chính xác thấp.
- VRAM thường là giới hạn đầu tiên khi chạy mô hình AI trên máy cá nhân.

97 Spectre và Meltdown

Năm 2018, người ta công bố các lỗ hổng nằm ngay trong kỹ thuật tăng tốc của CPU. Đây là bài học lớn về thiết kế.

1. **Thực thi suy đoán**
   CPU chạy trước theo dự đoán, sai thì hủy kết quả trong thanh ghi.

2. **Dấu vết còn lại**
   Dữ liệu đã nạp vào cache thì không bị hủy.

3. **Kênh kề**
   Đo thời gian truy cập để biết khối nào đang trong cache, từ đó suy ra dữ liệu.

Lỗ hổng	Lợi dụng	Giảm thiểu
Meltdown	Đọc suy đoán vùng nhớ kernel	Cách ly bảng trang kernel (KPTI), sửa trong phần cứng mới
Spectre	Lừa bộ dự đoán nhánh	Cập nhật vi mã, trình biên dịch, trình duyệt

Điều đáng nhớ: kết quả đúng chưa chắc an toàn. Thời gian chạy cũng là một kênh lộ thông tin.

- ✓ Luôn cập nhật hệ điều hành và BIOS, vì bản vá nằm ở đó.
- ✓ So sánh mật khẩu phải dùng hàm thời gian cố định, cùng ý tưởng.
- ✓ Tăng tốc và an toàn thường phải đánh đổi: một số bản vá làm chậm máy.

98 Chọn laptop cho sinh viên IT

Dùng kiến trúc máy tính để đọc cấu hình, đừng chỉ nhìn tên dòng máy. Đây là gợi ý theo nhu cầu, không phải quảng cáo.

Linh kiện	Nên chọn	Vì sao
RAM	16 GB, tốt nhất 2 kênh	IDE, Docker, trình duyệt ăn nhiều RAM
Ổ đĩa	SSD NVMe từ 512 GB	Đây là nâng cấp đáng giá nhất
CPU	Từ 6 lõi nếu biên dịch nhiều	Biên dịch, Docker chạy song song tốt
GPU rời	Chỉ khi học AI, đồ họa, game	Quan trọng nhất là VRAM
Tản nhiệt, pin	Đọc đánh giá thực tế	Máy nóng sẽ tự giảm xung nhịp

TRƯỚC KHI MUA
Xem RAM có nâng được không. Nhiều máy mỏng hàn RAM chết vào bo mạch.

MÁY ĐÃ CÓ
Thêm RAM, đổi sang SSD thường rẻ hơn mua máy mới.

Kiểm tra máy đang dùng	Lệnh
CPU, số lõi, cache	Task Manager, tab Performance. Linux: lscpu
RAM, số kênh	Xem Slots used. Linux: sudo dmidecode -t memory
Ổ đĩa	Windows: mục Storage. Linux: lsblk

Giá và cấu hình thay đổi nhanh. Hãy kiểm tra yêu cầu phần mềm của ngành bạn trước khi mua.

Bộ công cụ

Toàn bộ đều miễn phí. Không cần phần cứng đặc biệt, laptop nào cũng học được.

Công cụ	Dùng để	Ghi chú
MARS	Viết, chạy từng bước MIPS	Cần Java, nhiều trường dùng
mips.py	Chạy MIPS bằng Python	Kèm cẩm nang, phần lệnh hay dùng
NASM	Hợp dịch x86-64	Dùng trên Linux hoặc WSL
gcc, gdb, objdump	Biên dịch, debug, xem mã máy	sudo apt install build-essential gdb
Compiler Explorer	Xem hợp ngữ ngay trên web	godbolt.org, nhiều trình biên dịch
Logisim Evolution	Vẽ, mô phỏng mạch số	Hay dùng cho bài lab mạch

```
java -jar Mars4_5.jar                # mở MARS
python mips.py bai.asm               # chạy bằng bộ giả lập kèm theo
python mips.py --list bai.asm        # xem mã máy từng lệnh
python mips.py --stats bai.asm       # đếm số lệnh đã chạy
```

1 **Học MIPS**
Dùng MARS, bật chạy từng bước, xem bảng thanh ghi.

2 **Học x86-64**
Dùng WSL hoặc máy Linux, viết nhỏ rồi gọi từ C.

mips.py không có số thực và delayed branch. Bài trên lớp hãy kiểm tra lại bằng MARS, nếu trường bạn chấm bằng công cụ đó.

Lab 1: Tự viết CPU trong 25 dòng

Viết bộ giả lập là cách hiểu CPU sâu nhất. Bản này chạy được vòng lặp hợp ngữ thật.

```
def run(lines):
    labels = {x[:-1]: i for i, x in enumerate(lines) if x[-1] == ":"}
    reg, pc = {"$zero": 0}, 0
    while pc < len(lines):
        op, *a = lines[pc].replace(",", " ").split()    # nạp, giải mã
        pc += 1
        if op == "addi":                                # thực thi
            reg[a[0]] = reg.get(a[1], 0) + int(a[2])
        elif op == "add":
            reg[a[0]] = reg.get(a[1], 0) + reg.get(a[2], 0)
        elif op == "beq" and reg.get(a[0], 0) == reg.get(a[1], 0):
            pc = labels[a[2]]
        elif op == "j":
            pc = labels[a[0]]
    return reg

prog = ["addi $t0, $zero, 1", "addi $t1, $zero, 6", "loop:",
        "beq $t0, $t1, done", "add $t2, $t2, $t0",
        "addi $t0, $t0, 1", "j loop", "done:"]
print("Tổng 1 đến 5 =", run(prog)["$t2"])
```

KẾT QUẢ KHI CHẠY

```
Tổng 1 đến 5 = 15
```

BÀI TẬP

Thêm sub, slt, bne, lw, sw với bộ nhớ là một mảng. Đếm số lệnh mỗi loại, rồi tính CPI theo bảng số chu kỳ cho trước.

Lab 2: Sắp xếp bằng MIPS

Bài lab kinh điển: vòng lặp lồng nhau, mảng, so sánh và đổi chỗ.

```
        .data
a:      .word 5, 2, 9, 1, 7, 3
n:      .word 6
sp:     .asciiz " "
        .text
main:   la   $s0, a
        lw   $s1, n
        addi $s2, $s1, -1
outer:  blez $s2, print
        move $t0, $s0
        move $t1, $s2
inner:  blez $t1, next
        lw   $t2, 0($t0)
        lw   $t3, 4($t0)
        ble  $t2, $t3, skip
        sw   $t3, 0($t0)
        sw   $t2, 4($t0)
skip:   addi $t0, $t0, 4
        addi $t1, $t1, -1
        j    inner
next:   addi $s2, $s2, -1
        j    outer
print:  beqz $s1, end
        lw   $a0, 0($s0)
        li   $v0, 1
        syscall
        la   $a0, sp
        li   $v0, 4
        syscall
        addi $s0, $s0, 4
        addi $s1, $s1, -1
        j    print
end:    li   $v0, 10
        syscall
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
1 2 3 5 7 9
```

BÀI TẬP

Thêm cờ dừng sớm khi không có đổi chỗ. Viết phần đổi chỗ thành hàm swap(p) dùng jal. Đọc n và mảng từ bàn phím bằng syscall 5.

Lab 3: Đếm lệnh, tính CPI

Nối hai đầu của môn học: chạy chương trình thật, đếm từng loại lệnh rồi áp dụng công thức hiệu năng.

```
        .data
a:      .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
        .text
main:   la   $t0, a
        li   $t1, 10
        li   $t2, 0
loop:   lw   $t3, 0($t0)
        add  $t2, $t2, $t3
        addi $t0, $t0, 4
        addi $t1, $t1, -1
        bne  $t1, $zero, loop
        li   $v0, 10
        syscall
```

SỐ LỆNH ĐÃ CHẠY

```
Số lệnh đã chạy: 56
R=11 I=25 J=0 load=10 store=0 branch=10
```

```
count = {"R": 11, "I": 25, "load": 10, "branch": 10}  # vừa đo
cycles = {"R": 4, "I": 4, "load": 5, "branch": 3}     # giả định
total = sum(count[k] * cycles[k] for k in count)
n = sum(count.values())
print("Tổng chu kỳ:", total, "| số lệnh:", n)
print(f"CPI = {total / n:.2f}")
```

KẾT QUẢ KHI CHẠY

```
Tổng chu kỳ: 224 | số lệnh: 56
CPI = 4.00
```

BÀI TẬP

Viết lại vòng lặp theo cách chỉ số a[i], đếm lại số lệnh. Bạn tiết kiệm được bao nhiêu phần trăm khi dùng con trỏ?

Lab 4: Hàm strlen x86-64

Hàm thư viện thật ra chỉ là vài lệnh. Viết lại strlen và gọi từ C.

```
global my_strlen
section .text
my_strlen:                ; rdi = địa chỉ chuỗi
    xor eax, eax          ; độ dài = 0
.loop:
    cmp byte [rdi + rax], 0
    je .done              ; gặp byte 0 thì dừng
    inc rax
    jmp .loop
.done:
    ret
```

KẾT QUẢ KHI GỌI TỪ C

```
9
10
```

```
#include <stdio.h>
long long my_strlen(const char *s);

int main(void) {
    printf("%lld\n", my_strlen("Code4Life"));
    printf("%lld\n", my_strlen("Việt Nam"));
    return 0;
}
```

Chuỗi Việt Nam có 8 ký tự nhưng 10 byte: chữ ệ chiếm 3 byte trong UTF-8. strlen đếm byte, không đếm ký tự.

BÀI TẬP

Viết tiếp hàm đếm chữ a, đảo ngược chuỗi tại chỗ, tìm số lớn nhất trong mảng.

Lab 5: In số ra màn hình

Không có printf, phải tự đổi số thành chữ: chia 10 lấy dư, ghi từ cuối bộ đệm về đầu.

```
section .data
buf:    times 20 db 0
section .text
global _start
_start:
    mov rax, 2026
    mov rcx, buf + 19
    mov byte [rcx], 10
    mov rbx, 10
.digit:
    xor rdx, rdx
    div rbx             ; thương, dư
    add dl, '0'
    dec rcx
    mov [rcx], dl
    test rax, rax
    jnz .digit
    mov rax, 1          ; write
    mov rdi, 1
    mov rsi, rcx
    mov rdx, buf + 20
    sub rdx, rcx        ; độ dài
    syscall
    mov rax, 60
    xor rdi, rdi
    syscall
```

KẾT QUẢ CHẠY TRÊN LINUX

```
2026
```

BÀI TẬP

In số âm. In số ở hệ 16. Viết thành hàm print_int rồi dùng trong một vòng lặp in 1 đến 10.

Quên xor rdx, rdx trước div là lỗi kinh điển: div chia cả 128 bit rdx:rax, nên có thể báo lỗi và kết thúc chương trình.

Lab 6: Giả lập cache duyệt ma trận

Kiểm chứng câu đố ở bìa: khối lớn giúp duyệt hàng, không giúp duyệt cột. Cache đủ lớn thì cột cũng ổn.

```
def miss_rate(addrs, lines, block):        # cache ánh xạ trực tiếp
    tags, misses = [None] * lines, 0
    for a in addrs:
        blk = a // block
        if tags[blk % lines] != blk:
            tags[blk % lines] = blk
            misses += 1
    return misses / len(addrs)

N = 128                                    # ma trận int 64 KB
row = [(i * N + j) * 4 for i in range(N) for j in range(N)]
col = [(i * N + j) * 4 for j in range(N) for i in range(N)]

print("Cache 4 KB     | hàng  | cột")
for block in (16, 32, 64, 128):
    lines = 4096 // block
    print(f"Khối {block:3} byte | {miss_rate(row, lines, block):5.1%}"
          f" | {miss_rate(col, lines, block):5.1%}")
print(f"Cột, cache 64 KB: {miss_rate(col, 1024, 64):.1%}")
```

KẾT QUẢ KHI CHẠY

```
Cache 4 KB     | hàng  | cột
Khối  16 byte | 25.0% | 100.0%
Khối  32 byte | 12.5% | 100.0%
Khối  64 byte |  6.2% | 100.0%
Khối 128 byte |  3.1% | 100.0%
Cột, cache 64 KB: 6.2%
```

BÀI TẬP

Thử cache 2 đường, 4 đường. Vẽ biểu đồ tỉ lệ trượt theo dung lượng.

Ý tưởng đồ án môn học

Đồ án tốt có phần chạy được, có số đo và giải thích được bằng kiến thức môn học.

Đề tài	Mức	Kiến thức dùng
Game đoán số, máy tính bỏ túi bằng MIPS	Dễ	Syscall, rẽ nhánh, hàm
Thư viện chuỗi, sắp xếp bằng hợp ngữ	Dễ	Mảng, stack, quy ước gọi hàm
Giả lập cache có giao diện	Vừa	Ánh xạ, thay thế, ghi trễ
Giả lập pipeline MIPS	Vừa	Hazard, forwarding, stall
Hợp dịch MIPS ra mã máy	Vừa	Định dạng lệnh, nhãn, hai lượt
CPU trên Logisim	Khó	Datapath, tín hiệu điều khiển
CPU RISC-V trên FPGA	Khó	Verilog, pipeline
Tối ưu nhân ma trận	Khó	Cache, chia khối, SIMD, đa luồng

- ✓ Có số đo trước và sau, ghi rõ cấu hình máy, chạy nhiều lần lấy trung bình.
- ✓ Dùng Git từ ngày đầu, mỗi thành viên có commit của mình.
- ✓ Được dùng AI hỗ trợ thì phải hiểu từng dòng, vì vấn đáp sẽ hỏi tới.

Bài mẫu: số và IEEE 754

Đề 1. Biểu diễn -37 ở dạng bù 2, 8 bit.

Lời giải. 37 = 00100101. Đảo bit: 11011010. Cộng 1: 11011011.

Đề 2. Cộng 8 bit bù 2: 01100100 + 01010000. Có tràn không?

Lời giải. 100 + 80 = 180, vượt 127. Kết quả 10110100 mang dấu âm. Hai số dương ra âm nên tràn.

Đề 3. Đổi 12,375 sang float 32 bit, viết dạng hex.

Lời giải. 12,375 = 1100,011 = 1,100011 × 2^3. S = 0, e = 3 + 127 = 130 = 10000010, f = 100011 rồi thêm số 0.

Đề 4. Giá trị của float 0xC1200000 là bao nhiêu?

Lời giải. S = 1, e = 10000010 = 130, f = 0100... Giá trị = -1,01 × 2^3 = -1010 = -10.

```
import struct

print(format(-37 & 0xFF, "08b"))
print(format(100 + 80, "08b"))
print(struct.pack(">f", 12.375).hex().upper())
print(struct.unpack(">f", bytes.fromhex("C1200000"))[0])
```

KẾT QUẢ KHI CHẠY

```
11011011
10110100
41460000
-10.0
```

Làm xong hãy thử ngược lại: đổi kết quả về số ban đầu để tự kiểm tra.

Bài mẫu: dịch C sang MIPS

Đề. Dịch sang MIPS: for (i = 0; i < n; i++) if (A[i] < 0) A[i] = -A[i]. Biết $s0 là địa chỉ A, $s1 là n.

```
        .data
A:      .word 3, -4, 5, -6
        .text
main:   la   $s0, A
        li   $s1, 4
        li   $t0, 0             # i = 0
for:    slt  $t1, $t0, $s1      # i < n ?
        beq  $t1, $zero, end
        sll  $t2, $t0, 2        # i × 4
        add  $t2, $t2, $s0      # địa chỉ A[i]
        lw   $t3, 0($t2)
        slt  $t4, $t3, $zero    # A[i] < 0 ?
        beq  $t4, $zero, next
        sub  $t3, $zero, $t3    # đổi dấu
        sw   $t3, 0($t2)
next:   addi $t0, $t0, 1        # i++
        j    for
end:    lw   $t5, 4($s0)
        lw   $t6, 12($s0)
        li   $v0, 10
        syscall
# Xem: $t5 $t6
```

KẾT QUẢ CHẠY BỘ GIẢ LẬP MIPS

```
$t5=4   $t6=6
```

Bước 1. Gán thanh ghi cho từng biến, ghi ra trước khi viết.

Bước 2. Viết khung vòng lặp: nhãn, kiểm tra, tăng i, nhảy về.

Bước 3. Viết thân: tính địa chỉ, nạp, xử lý, lưu.

Trong bài thi chỉ cần phần từ nhãn for tới end. Phần còn lại ở đây để chạy thử.

Bài mẫu: mã hóa lệnh

Đề 1. Mã hóa sub $t2, $s0, $t1.

Lời giải. Kiểu R: op = 0, rs = $s0 = 16, rt = $t1 = 9, rd = $t2 = 10, shamt = 0, funct = 34. Ghép bit: 000000 10000 01001 01010 00000 100010.

Đề 2. Giải mã 0x8D2A0004.

Lời giải. 6 bit đầu 100011 = 35 là lw. rs = 01001 = $t1, rt = 01010 = $t2, hằng số 4. Vậy lw $t2, 4($t1).

Đề 3. Lệnh beq ở địa chỉ 0x00400020, nhảy tới 0x00400008. Trường hằng số bằng bao nhiêu?

Lời giải. (0x00400008 - 0x00400024) / 4 = -7, dạng 16 bit là 0xFFF9.

ĐỊA CHỈ, MÃ MÁY VÀ LỆNH

```
0x00400000  02095022  sub $t2, $s0, $t1
0x00400004  8d2a0004  lw $t2, 4($t1)
```

```
def decode(w):
    return dict(op=w >> 26, rs=w >> 21 & 31, rt=w >> 16 & 31,
                rd=w >> 11 & 31, imm=w & 0xFFFF, funct=w & 63)

print(decode(0x8D2A0004))
print(hex((0x00400008 - 0x00400024) // 4 & 0xFFFF))
```

KẾT QUẢ KHI CHẠY

```
{'op': 35, 'rs': 9, 'rt': 10, 'rd': 0, 'imm': 4, 'funct': 4}
0xfff9
```

Thứ tự trường trong mã máy là rs, rt, rd, khác thứ tự viết rd, rs, rt. Đây là chỗ mất điểm nhiều nhất.

Bài mẫu: hiệu năng

Đề 1. Chương trình 2 tỉ lệnh, máy 2,5 GHz. Lệnh ALU 50% CPI 1, Load 30% CPI 4, Branch 20% CPI 2. Tính CPI và thời gian.

Lời giải. CPI = 0,5 × 1 + 0,3 × 4 + 0,2 × 2 = 2,1. Thời gian = $2 \times 10^9 \times 2{,}1 / 2{,}5 \times 10^9$ = 1,68 giây.

Đề 2. Trình biên dịch mới bớt 20% số lệnh nhưng CPI tăng lên 2,4. Có nhanh hơn không?

Lời giải. Tỉ lệ thời gian = 0,8 × 2,4 / 2,1 = 0,914. Nhanh hơn khoảng 1,09 lần.

Đề 3. Phần nhân chiếm 60% thời gian. Tăng tốc phần đó 4 lần thì cả chương trình nhanh bao nhiêu?

Lời giải. 1 / (0,4 + 0,6 / 4) = 1 / 0,55 = 1,82 lần.

```
cpi = .5 * 1 + .3 * 4 + .2 * 2
print(round(cpi, 2), round(2e9 * cpi / 2.5e9, 2))
print(round(.8 * 2.4 / cpi, 3), round(cpi / (.8 * 2.4), 2))
print(round(1 / (.4 + .6 / 4), 2))
```

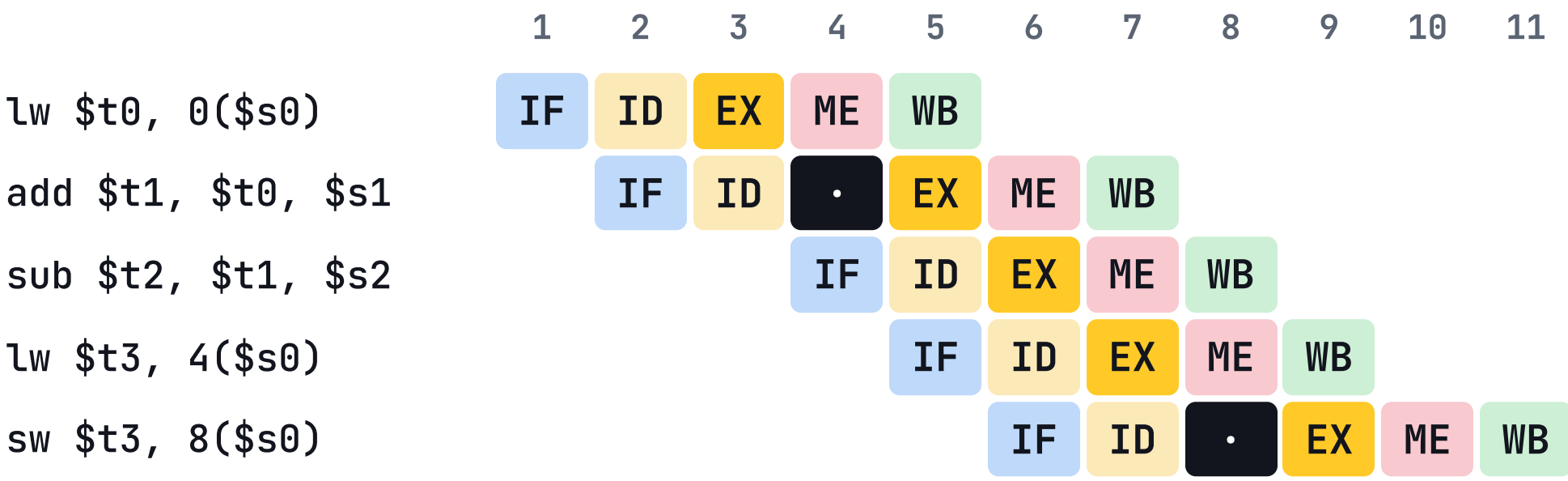
KẾT QUẢ KHI CHẠY

```
2.1 1.68
0.914 1.09
1.82
```

Ghi rõ công thức trước khi thay số. Sai số học vẫn được một phần điểm, không có công thức thì khó được điểm.

Bài mẫu: pipeline

Đề. Tìm xung đột và tính số chu kỳ của đoạn code sau, pipeline 5 tầng có chuyển tiếp.



KẾT QUẢ KHI CHẠY

```
Tổng 11 chu kỳ, 2 stall, CPI = 2.2
```

Lệnh 2. Cần $t0 ngay sau lw: xung đột load-use, chèn 1 stall rồi chuyển tiếp từ MEM/WB.

Lệnh 3. Cần $t1 của lệnh add: chuyển tiếp từ EX/MEM, không cần stall.

Lệnh 5. sw cần $t3 ngay sau lw. Mô hình đơn giản ở đây tính 1 stall.

Công thức. Số chu kỳ = 5 + (5 - 1) + 2 stall = 11.

Một số giáo trình cho chuyển tiếp từ MEM sang MEM với sw, khi đó lệnh 5 không cần stall. Hãy theo mô hình đề bài cho.

Bài mẫu: cache

Đề 1. Cache 64 KB, khối 32 byte, kết hợp 4 đường, địa chỉ 32 bit. Tính tag, index, offset.

Lời giải. Số dòng = $2^{16} / 2^5 = 2^{11}$. Số tập = $2^{11} / 4 = 2^9$. Offset 5, index 9, tag 32 - 9 - 5 = 18.

Đề 2. L1 trúng mất 1 chu kỳ, tỉ lệ trượt 4%, L2 trúng mất 10 chu kỳ, tỉ lệ trượt 25%, RAM mất 120 chu kỳ. Tính AMAT.

Lời giải. AMAT = 1 + 0,04 × (10 + 0,25 × 120) = 1 + 0,04 × 40 = 2,6 chu kỳ.

Đề 3. CPI gốc 1, 30% lệnh truy cập dữ liệu, cache lệnh trượt 1%, cache dữ liệu trượt 5%, phạt 80 chu kỳ. Tính CPI.

Lời giải. CPI = 1 + 0,01 × 80 + 0,3 × 0,05 × 80 = 1 + 0,8 + 1,2 = 3.

```
from math import log2

lines = 64 * 1024 // 32
sets = lines // 4
print(int(log2(32)), int(log2(sets)), 32 - int(log2(sets)) - 5)
print(round(1 + .04 * (10 + .25 * 120), 2))
print(round(1 + .01 * 80 + .3 * .05 * 80, 2))
```

KẾT QUẢ KHI CHẠY

```
5 9 18
2.6
3.0
```

Đề thường cho dung lượng KB, MB. Đổi hết về lũy thừa của 2 rồi trừ số mũ sẽ nhanh và ít sai.

Bảng công thức

Nội dung	Công thức
Thời gian CPU	Số lệnh × CPI ÷ tần số
CPI trung bình	Σ tỉ lệ lệnh × CPI của loại đó
Amdahl	1 / ((1 - p) + p / s)
Công suất động	C × V^2 × f
Bù 2 n bit	-2^{n-1} đến $2^{n-1} - 1$
float	$(-1)^s$ × 1,f × 2^(e - 127)
double	$(-1)^s$ × 1,f × 2^(e - 1023)
Địa chỉ a[i]	Nền + i × cỡ phần tử
Đích của beq	PC + 4 + độ dời × 4
Đích của j	4 bit cao của PC + 4, ghép 26 bit, ghép 00
Pipeline k tầng, n lệnh	k + n - 1 + số stall
CPI có nhánh	1 + tỉ lệ nhánh × tỉ lệ đoán sai × phạt
Cache	Offset = $\log_2$(khối), index = $\log_2$(số tập)
Tag	Số bit địa chỉ - index - offset
Tổng bit cache	Số dòng × (dữ liệu + tag + valid)
AMAT	Hit + miss × phạt
CPI có cache	Gốc + miss lệnh × phạt + tỉ lệ nạp lưu × miss dữ liệu × phạt
Băng thông RAM	MT/s × 8 byte × số kênh

Chụp lại trang này để xem nhanh trước giờ thi. Một số trường cho mang một tờ tài liệu, hãy hỏi giảng viên.

Tự luyện

Tự làm trước, đáp án ở trang sau.

Bài 1. Biểu diễn −100 ở dạng bù 2, 8 bit, rồi đổi sang hex.

Bài 2. Đổi −0,625 sang float 32 bit, viết dạng hex.

Bài 3. Mã hóa lệnh addi $t1, $t0, −8 dạng hex.

Bài 4. Giải mã 0x01095020.

Bài 5. Máy A: 3 GHz, CPI 1,5. Máy B: 2 GHz, CPI 0,9. Cùng số lệnh, máy nào nhanh hơn, bao nhiêu lần?

Bài 6. 75% chương trình song song được. Dùng 8 lõi thì nhanh bao nhiêu lần? Tối đa bao nhiêu lần?

Bài 7. Pipeline 5 tầng chạy 200 lệnh, có 30 stall. Tính số chu kỳ và CPI.

Bài 8. Cache trực tiếp 32 KB, khối 64 byte, địa chỉ 32 bit. Tính offset, index, tag và tổng số bit.

Bài 9. Hit 2 chu kỳ, tỉ lệ trượt 8%, phạt 100 chu kỳ. Tính AMAT.

Bài 10. Tính băng thông đỉnh của DDR4-2666 hai kênh theo GB/s.

Đáp án tự luyện

Các đáp án dưới đây do code tính ra.

KẾT QUẢ KHI CHẠY

```
1. 10011100 0x9c
2. BF200000
3. 2109FFF8
4. op 0 rs 8 rt 9 rd 10 funct 32 = add $t2, $t0, $t1
5. B nhanh hơn 1.11 lần
6. 2.91 lần, tối đa 4.0 lần
7. 234 chu kỳ, CPI 1.17
8. offset 6, index 9, tag 17, tổng 271360 bit
9. 10.0 chu kỳ
10. 42.7 GB/s
```

Mẹo kiểm tra lại	Cách làm
Bù 2	Cộng số với bản đổi dấu phải ra 0
IEEE 754	Giải mã ngược về số thập phân
Mã hóa lệnh	Tổng số bit các trường phải bằng 32
Cache	Tag + index + offset phải bằng số bit địa chỉ
Amdahl	Kết quả không vượt quá 1 / (1 - p)

Trắc nghiệm (1/2)

1. Số -1 ở dạng bù 2, 8 bit là gì?
A. 10000001 B. 11111110 C. 11111111 D. 00000001

2. 0x2A bằng bao nhiêu?
A. 40 B. 42 C. 44 D. 52

3. Cộng hai số dương ra số âm nghĩa là gì?
A. Kết quả đúng B. Tràn số có dấu C. Lỗi phần cứng D. Không thể xảy ra

4. float 32 bit có bao nhiêu bit số mũ?
A. 23 B. 11 C. 8 D. 7

5. Bias của float 32 bit là bao nhiêu?
A. 127 B. 128 C. 255 D. 1023

6. Cổng nào đủ để tạo ra mọi mạch logic?
A. XOR B. NAND C. NOT D. AND

7. Thanh ghi MIPS nào luôn bằng 0?
A. $at B. $v0 C. $sp D. $zero

8. Lệnh nào lưu địa chỉ trở về vào $ra?
A. j B. jr C. jal D. beq

9. Phần tử a[5] của mảng word có độ dời bao nhiêu byte?
A. 5 B. 10 C. 20 D. 40

10. Lệnh add thuộc định dạng nào?
A. R B. I C. J D. Giả lệnh

Trắc nghiệm (2/2)

11. Lệnh lw thuộc định dạng nào?
A. R B. I C. J D. Giả lệnh

12. Hàm con phải giữ nguyên thanh ghi nào?
A. $t0 B. $a0 C. $s0 D. $v0

13. Trên Linux x86-64, tham số đầu tiên nằm ở đâu?
A. rax B. rdi C. rcx D. rsp

14. 10^9 lệnh, CPI 2, xung nhịp 2 GHz thì chạy mất bao lâu?
A. 0,5 giây B. 1 giây C. 2 giây D. 4 giây

15. 50% chương trình song song được, tăng tốc tối đa là bao nhiêu?
A. 2 lần B. 4 lần C. 5 lần D. Vô hạn

16. Pipeline 5 tầng, 100 lệnh, không xung đột thì mất bao nhiêu chu kỳ?
A. 100 B. 104 C. 105 D. 500

17. Xung đột load-use có chuyển tiếp thì còn bao nhiêu stall?
A. 0 B. 1 C. 2 D. 3

18. Trường nào của địa chỉ chọn dòng cache?
A. Tag B. Index C. Offset D. Valid

19. Cache trực tiếp 16 KB, khối 64 byte có bao nhiêu dòng?
A. 64 B. 128 C. 256 D. 512

20. DMA có tác dụng gì?
A. Tăng xung nhịp B. Thiết bị tự chép dữ liệu vào RAM C. Giảm cache D. Đổi số thực

Đáp án trắc nghiệm

Câu	Đáp án	Giải thích
1	C	Đảo 00000001 rồi cộng 1
2	B	2 × 16 + 10
3	B	Vượt phạm vi của số có dấu
4	C	1 dấu, 8 mũ, 23 định trị
5	A	$2^7 - 1$
6	B	NAND là cổng vạn năng, NOR cũng vậy
7	D	Ghi vào cũng không đổi
8	C	jump and link
9	C	5 × 4
10	A	Ba thanh ghi
11	B	Có hằng số độ dời 16 bit
12	C	s là saved
13	B	rdi, rsi, rdx, rcx, r8, r9
14	B	$10^9 \times 2 / (2 \times 10^9)$
15	A	1 / (1 - 0,5)
16	B	5 + 100 - 1
17	B	Dữ liệu có sau tầng MEM
18	B	Tag để so sánh, offset chọn byte
19	C	$2^{14} / 2^6 = 2^8$
20	B	CPU chỉ ra lệnh, xong thì nhận ngắt

20 câu vấn đáp (1/2)

1. ISA và vi kiến trúc khác nhau thế nào?
ISA là tập lệnh phần mềm nhìn thấy. Vi kiến trúc là cách chip hiện thực nó. Intel, AMD khác vi kiến trúc nhưng cùng x86-64.

2. Vì sao dùng số bù 2?
Chỉ có một số 0, và cộng, trừ dùng chung một mạch.

3. Vì sao 0.1 + 0.2 khác 0.3?
0,1 là số vô hạn tuần hoàn trong nhị phân, nên bị làm tròn khi lưu.

4. Little endian là gì?
Byte thấp nằm ở địa chỉ thấp. x86 dùng little endian, giao thức mạng dùng big endian.

5. RISC khác CISC thế nào?
RISC ít lệnh đơn giản, độ dài cố định, chỉ load, store chạm bộ nhớ. CISC lệnh phức tạp, độ dài thay đổi.

6. Điều gì xảy ra khi gọi hàm?
Tham số vào thanh ghi hoặc stack, địa chỉ trở về được lưu, hàm cấp khung stack, xong thì khôi phục và quay lại.

7. $t và $s khác nhau thế nào?
$t có thể bị hàm con đổi. $s hàm con phải trả lại nguyên vẹn.

8. Vì sao pipeline tăng tốc?
Nhiều lệnh ở các tầng khác nhau. Độ trễ một lệnh không giảm, nhưng thông lượng tăng.

9. Ba loại xung đột pipeline?
Cấu trúc, dữ liệu, điều khiển.

10. Forwarding là gì?
Đưa kết quả từ thanh ghi pipeline về đầu vào ALU, không cần chờ ghi vào tệp thanh ghi.

20 câu vấn đáp (2/2)

11. Vì sao load-use vẫn phải stall?
Dữ liệu có sau tầng MEM, trong khi lệnh sau cần ở đầu tầng EX, nên trễ một chu kỳ.

12. Dự đoán nhánh để làm gì?
Nạp tiếp lệnh mà không phải chờ kết quả nhánh. Đoán sai thì xóa đường ống.

13. Vì sao có cache?
RAM chậm hơn CPU rất nhiều. Tính cục bộ giúp một bộ nhớ nhỏ, nhanh phục vụ đa số truy cập.

14. Ba loại trượt cache?
Bắt buộc, dung lượng, xung đột.

15. Ghi xuyên và ghi trễ khác nhau thế nào?
Ghi xuyên ghi cả xuống RAM. Ghi trễ chỉ ghi cache, khi khối bị đuổi mới ghi xuống.

16. Vì sao duyệt mảng theo hàng nhanh hơn?
Phần tử kề nhau nằm chung một khối cache, một lần trượt dùng được nhiều phần tử.

17. False sharing là gì?
Hai lõi ghi hai biến khác nhau trong cùng khối cache, khối bị chuyền qua lại liên tục.

18. SIMD là gì?
Một lệnh xử lý nhiều dữ liệu cùng lúc, ví dụ AVX cộng 8 số float.

19. CPU khác GPU thế nào?
CPU tối ưu độ trễ một luồng. GPU tối ưu thông lượng của rất nhiều luồng đơn giản.

20. Vì sao không tăng mãi xung nhịp?
Công suất tăng theo $V^2 \times f$, chip nóng quá mức tản nhiệt. Vì vậy ngành chuyển sang nhiều lõi.

Lỗi hay mất điểm

Nhầm lẫn	Cách đúng
Quên số 1 ẩn trong IEEE 754	Giá trị là 1,f chứ không phải 0,f
Quên cộng, trừ bias	e = mũ thật + 127
Viết nhầm thứ tự trường	Mã máy là rs, rt, rd
Độ dời beq tính từ PC	Tính từ PC + 4, đơn vị là lệnh
Offset mảng quên nhân 4	Bộ nhớ đánh địa chỉ theo byte
Pipeline tính n × k chu kỳ	k + n − 1 + stall
Có forwarding là hết stall	Load-use vẫn còn 1 stall
Dùng cỡ cache làm số dòng	Số dòng = dung lượng ÷ cỡ khối
Index tính theo số dòng khi có nhiều đường	Index tính theo số tập
Nhầm KB và Kb	Chú ý byte hay bit

Đề luôn có quy ước riêng: địa chỉ theo byte hay từ, có delayed branch không, có tính thời gian tra TLB không. Khoanh các quy ước này trước khi làm.

Kế hoạch 15 tuần

Tuần	Nội dung	Làm
1	Tổng quan, hiệu năng	Bài CPI, Amdahl
2	Số nguyên, phép toán bit	Đổi cơ số, bù 2
3	Số thực IEEE 754	Đổi tay, kiểm tra bằng code
4	Mạch số, ALU	Bài Logisim nếu có
5	MIPS: lệnh, nhánh, vòng lặp	Cài MARS, viết vòng lặp
6	MIPS: mảng, chuỗi	Lab 2
7	MIPS: hàm, stack, đệ quy	Ôn giữa kỳ
8	Mã hóa lệnh	Bài mã hóa, Lab 1
9	x86-64, gọi từ C	Lab 4, Lab 5
10	Datapath, điều khiển	Vẽ lại hình, thuộc bảng tín hiệu
11	Pipeline, xung đột	Bài mẫu pipeline
12	Cache cơ bản	Giả lập trực tiếp
13	Cache nâng cao, AMAT	Lab 6, bài mẫu cache
14	Vào ra, đa lõi, SIMD	Đo false sharing, SIMD
15	Ôn tập	Bài mẫu, trắc nghiệm, vấn đáp

Trình tự theo cẩm nang. Hãy chỉnh cho khớp đề cương và lịch của trường bạn.

Học gì tiếp theo?

Kiến trúc máy tính là nền của hệ điều hành, trình biên dịch, nhúng, bảo mật và tối ưu AI.

Hướng	Cần đào sâu
Lập trình hiệu năng cao	Cache, SIMD, đo bằng perf
Nhúng, IoT	ARM Cortex-M, ngắt, thanh ghi thiết bị
Thiết kế chip	Verilog, FPGA, RISC-V
An toàn thông tin	Đọc hợp ngữ, phân tích mã độc, khai thác lỗi có phép
AI hệ thống	GPU, CUDA, tối ưu bộ nhớ

1. **Nand2Tetris**
   Tự xây máy tính từ cổng NAND tới hệ điều hành nhỏ.

2. **CS:APP và các bài lab**
   Bomb lab, cache lab rất nổi tiếng, hợp để luyện đọc mã máy.

3. **Tự làm CPU RISC-V**
   Viết bằng Verilog, chạy mô phỏng hoặc trên FPGA.

4. **Đo code của chính bạn**
   Dùng perf, valgrind, Compiler Explorer để hiểu chương trình thật.

Nên học cùng cẩm nang Hệ điều hành, Cấu trúc dữ liệu và An toàn thông tin của Code4Life. Các môn này hỗ trợ nhau rất nhiều.

Tài liệu nên đọc

Nguồn	Vì sao nên đọc
Patterson, Hennessy **Computer Organization and Design**	Giáo trình gốc của môn học
Hennessy, Patterson **Computer Architecture: A Quantitative Approach**	Nâng cao, sau đại học
Bryant, O'Hallaron **Computer Systems: A Programmer's Perspective**	Góc nhìn của lập trình viên
Nisan, Schocken **The Elements of Computing Systems**	Tự xây máy tính
Tài liệu MARS	Syscall, giả lệnh
Tài liệu NASM, Intel SDM	Cú pháp, tập lệnh x86
Agner Fog: Optimization manuals	Tối ưu ở mức vi kiến trúc
godbolt.org	Xem hợp ngữ ngay trên web

Thấy chỗ nào chưa đúng? Anh em comment giúp để Code4Life sửa ở bản sau nhé.

Biên soạn: Tào Việt Đức, Code4Life Technology, với sự hỗ trợ của AI. Code MIPS chạy bằng bộ giả lập mips.py, hợp ngữ x86-64 chạy thật trên Linux qua WSL 2, số đo thực hiện trên Core i5-1035G1. Thông số trong ví dụ tính toán là giả định để minh họa.